GeoBrix Beta: User Guide

GeoBrix - The Building Blocks of Spatial Intelligence!

Primary Contact: Michael Johns | mjohns@databricks.com | Geospatial Specialist Leader [AMER]
Last Modified: Dec 2, 2025

gbx_rst_h3_rastertogridmin(tile, resolution)
gbx_rst_h3_tessellate(tile, resolution)
gbx_rst_height(tile)
gbx_rst_initnodata(tile)
gbx_rst_isempty(tile)
gbx_rst_maketiles(tile, size_in_mb)
gbx_rst_mapalgebra(tiles, expression)
gbx_rst_max(tile)
gbx_rst_median(tile)
gbx_rst_memsize(tile)
gbx_rst_merge(tiles)
gbx_rst_merge_agg(tile)
gbx_rst_metadata(tile)
gbx_rst_min(tile)
gbx_rst_ndvi(tile, red_band, nir_band)
gbx_rst_numbands(tile)
gbx_rst_pixelcount(tile)
gbx_rst_pixelheight(tile)
gbx_rst_pixelwidth(tile)
gbx_rst_rastertoworldcoord(tile, pixel_x, pixel_y)
gbx_rst_rastertoworldcoordx(tile pixel_x, pixel_y)
gbx_rst_rastertoworldcoordy(tile pixel_x, pixel_y)
gbx_rst_retile(tile, tile_width, tile_height)
gbx_rst_rotation(tile)
gbx_rst_scalex(tile)
gbx_rst_scaley(tile)
gbx_rst_separatebands(tile)
gbx_rst_skewx(tile)
gbx_rst_skewy(tile)
gbx_rst_srid(tile)
gbx_rst_subdatasets(tile)
gbx_rst_summary(tile)
gbx_rst_tooverlappingtiles(tile, tile_width, tile_height, overlap)
gbx_rst_transform(tile, target_srid)
gbx_rst_tryopen(tile)
gbx_rst_type(tile)
gbx_rst_updatetype(tile, new_type)
gbx_rst_upperleftx(tile)
gbx_rst_upperlefty(tile)
gbx_rst_width(tile)
gbx_rst_worldtorastercoord(tile, world_x, world_y)

Disclaimer

GeoBrix is not a product capability; it is a Geospatial SME group initiative out of our Field Engineering. It is provided for optional use by our partners and customers on Databricks, with no guarantees of future development or improvements. Since it is not part of the product, you cannot file Product support requests or expect any formal SLAs. However, you can reach out to the author(s) to informally explore improvements and offer feedback.

Distribution

For the Beta, the artifacts are sent to each interested customer for evaluation and feedback. There is not yet a public repository for customers to self-serve.

Release Notes

v2 DEC 02, 2025

- Altered python bindings to `databricks.labs.gbx.*` to more closely match scala bindings (was `geobrix.*`).
- Changed init script to install numpy 2.x which allows GDAL python array operations to successfully execute.
- A handful of functions known to throw exceptions during execution have been addressed by the changes to the init script: gbx_rst_mapalgebra(tiles, expression) and gbx_rst_ndvi(tile, red_band, nir_band).
- Added more error handling to bubble up exceptions during execution; functions now return null or a catch-all default with error messages captured.

v1 OCT 29, 2025

Initial Release

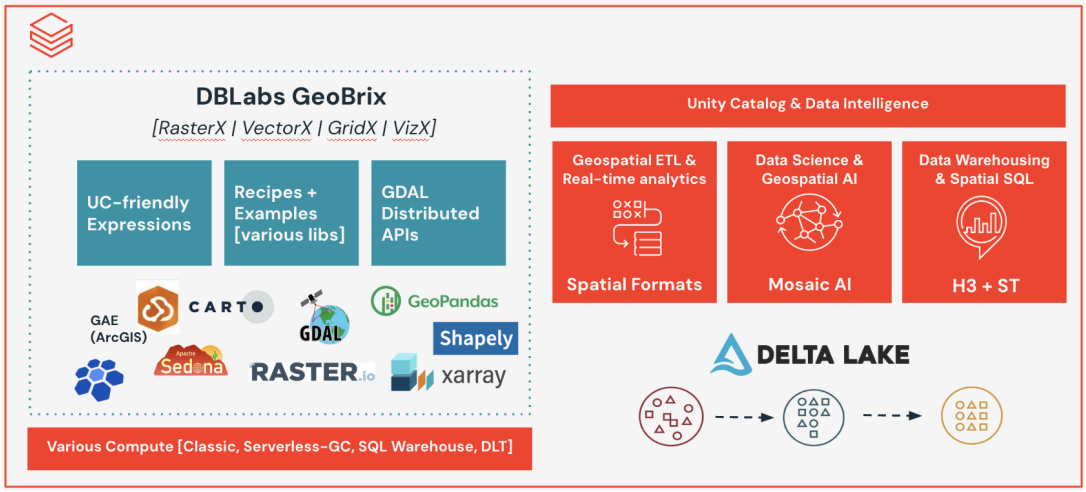
Known Limitations

- The Beta does not yet support Databricks Spatial Types directly but is standardized to WKB or WKT where geometries are involved. In addition to content in this guide, the provided notebooks, e.g. Shapefile Reader, have examples of converting to our built-in GEOMETRY type and using our built-in ST Geospatial Functions.
- A handful of functions are not yet ported. For raster: rst_dtmfromgeoms and for vector: st_interpolateelevation and st_triangulate.
- Spatial KNN is not yet ported; neither is H3 support for Geometry-based K-Ring and K-Loop.
- Custom Gridding is not fully ported.

Overview

Now that product built-in spatial sql functions have reached public preview as of DBR17.1, we are seeking to deliver the next generation of product-augmenting capabilities to help our customers adopt Databricks as their Geospatial Intelligence Platform of choice. GeoBrix project is a streamlined iteration to the existing, and quite popular, DBLabs Mosaic project. Beyond just porting existing Mosaic code, GeoBrix is modernized with expressions designed to work with our Data Intelligence Platform. GeoBrix will be a combination of heavy-weight (e.g. JAR) as well as lightweight (e.g Python, SQL) code artifacts. It also will focus on techniques to use the Databricks platform more widely.

With Databricks first having acquired MosaicML and now having made a product line, Mosaic AI, it has become clear that the DBLabs Mosaic project, sharing the name, needs to be revamped in name as well as any existing Mosaic capabilities that compete with product investments. If this were not the case, we would have simply iterated on DBLabs Mosaic "in-place" keeping the same name for what is now called GeoBrix. DBLabs Mosaic is in maintenance mode. The latest/last version targets DBR 13.3 LTS since product introduced ST functions starting with DBR 14. As such, Mosaic does not have any awareness of advancements in recent runtimes, including product support for spatial sql and types, and will be retired with DBR 13.3 EoS in AUG 2026.



GDAL Distributed API

The provided JAR contains "heavy-weight" Scala APIs, including implemented functions and readers, especially those powered by GDAL.

VectorX

Refactor of select Mosaic vector functions that augment existing product ST Geospatial Functions. Right now, this only includes a single function to handle updating existing Mosaic geometry data to those supported by product, so that users do not need to install (older) Mosaic in order to get to using the latest spatial features. There are a couple of more advanced functions that are also candidates for (eventual) porting, not yet available in product: ST_InterpolateElevation and ST_Triangulate.

Refactor of Mosaic discrete global grid indexing functions that augment existing product [H3 Geospatial Functions] as well as functions for other grid systems.

- **H3** - Nothing ported yet. Only a handful of geometry-aware functions would be candidates.
- **BNG** - Ported for Great Britain customers.
- **Custom Gridding** - Partially ported for ad-hoc gridding.

Refactor and improvement of Mosaic raster functions. Product does not yet support anything for raster, so this is a “fully” gap-filling capability.

Installation

As of OCT 2025, we are still in process of establishing the new DBLabs GitHub repository for the GeoBrix project. The code artifacts are delivered currently as a ZIP file, which includes the following under **geobrix-0.1.0-beta** folder:

1. **geobrix-gdal-init.sh** - init script
2. **dblabs_geobrix-0.1.0-py3-none-any.whl** - python API (requires JAR for GDAL)
3. **geobrix-0.1.0-jar-with-dependencies.jar** - heavy-weight API and readers (via init script)
4. **libgdalalljni.so** - native JNI bindings for JAR

Note: There are also notebooks under folder **examples.** These include the following:

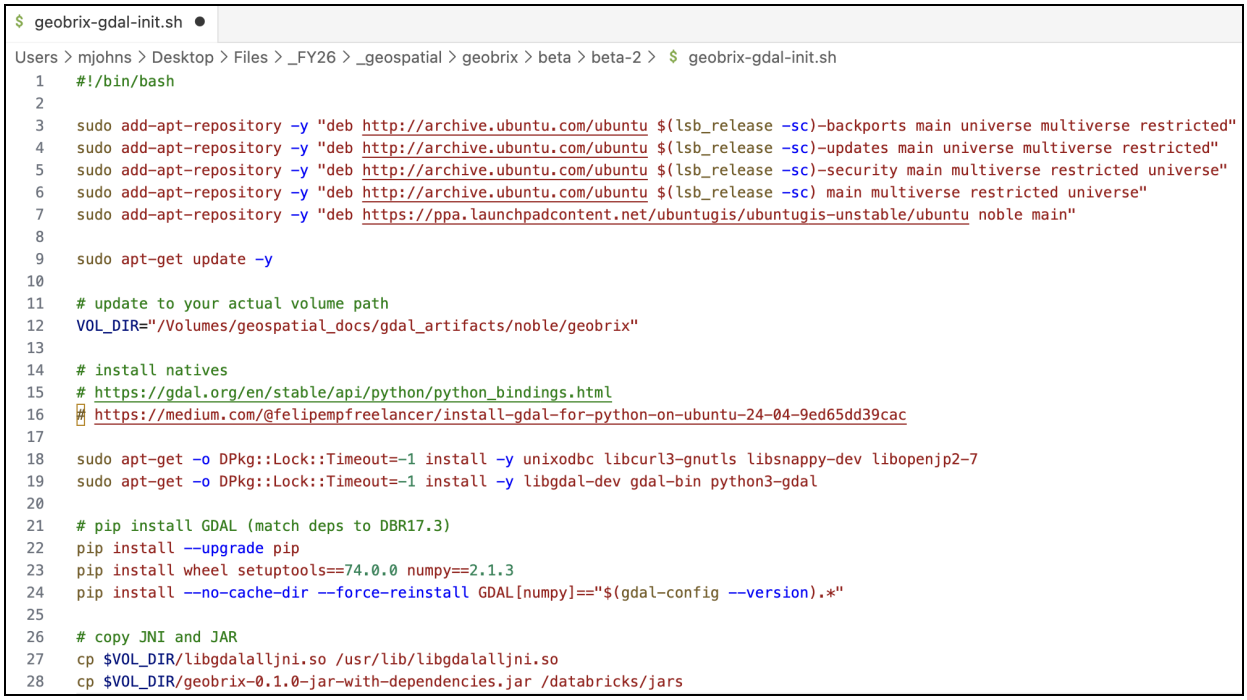
- Registered Functions - *GeoBrix - SQL Functions.dbc*
- Vector Readers **- ogr-reader-examples** folder; also see **ogr-resources** sub-folder
  - **Shapefile:** *VectorX - Shapefile Reader Example.dbc*; also, *download_census_shapefiles.py.dbc*
  - **GeoJSON**: *VectorX - GeoJSON Reader Example.dbc*
  - **GeoPackage**: *VectorX - GPKG Reader Example.dbc*
  - **File GeoDatabase:** *VectorX - FileGDB Reader Example.dbc*

- Raster - **eo-series** folder
    - *01. Search STACs.dbc*
    - *02. Download STACs.dbc*
    - *03. Gridded EO Data.dbc*
    - *04. Band Stacking + Clipping.dbc*
    - Also *config_nb.dbc* and *library.py (file not notebook)*

[1] Init Script Config

The provided init script will require some customization prior to uploading to Volumes and adding to your Cluster.

First, note the section that reads: # update to your actual volume path. Determine where you are going to store the artifacts in Volumes and change this section to match. You will see that the JAR and JNI `.so` (shared object) are copied onto the cluster through the Init Script. If the version of the JAR changes, you will need to modify this script.



$ geobrix-gdal-init.sh ●

Users > mjohns > Desktop > Files > _FY26 > _geospatial > geobrix > beta > beta-2 > $ geobrix-gdal-init.sh

```
#!/bin/bash

sudo add-apt-repository -y "deb http://archive.ubuntu.com/ubuntu $(lsb_release -sc)-backports main universe multiverse restricted"
sudo add-apt-repository -y "deb http://archive.ubuntu.com/ubuntu $(lsb_release -sc)-updates main universe multiverse restricted"
sudo add-apt-repository -y "deb http://archive.ubuntu.com/ubuntu $(lsb_release -sc)-security main multiverse restricted universe"
sudo add-apt-repository -y "deb http://archive.ubuntu.com/ubuntu $(lsb_release -sc) main multiverse restricted universe"
sudo add-apt-repository -y "deb https://ppa.launchpadcontent.net/ubuntugis/ubuntugis-unstable/ubuntu noble main"

sudo apt-get update -y

# update to your actual volume path
VOL_DIR="/Volumes/geospatial_docs/gdal_artifacts/noble/geobrix"

# install natives
# https://gdal.org/en/stable/api/python/python_bindings.html
# https://medium.com/@felipempfreelancer/install-gdal-for-python-on-ubuntu-24-04-9ed65dd39cac

sudo apt-get -o DPkg::Lock::Timeout=-1 install -y unixodbc libcurl3-gnutls libsnappy-dev libopenjp2-7
sudo apt-get -o DPkg::Lock::Timeout=-1 install -y libgdal-dev gdal-bin python3-gdal

# pip install GDAL (match deps to DBR17.3)
pip install --upgrade pip
pip install wheel setuptools==74.0.0 numpy==2.1.3
pip install --no-cache-dir --force-reinstall GDAL[numpy]=="$(gdal-config --version).*"

# copy JNI and JAR
cp $VOL_DIR/libgdalalljni.so /usr/lib/libgdalalljni.so
cp $VOL_DIR/geobrix-0.1.0-jar-with-dependencies.jar /databricks/jars
```

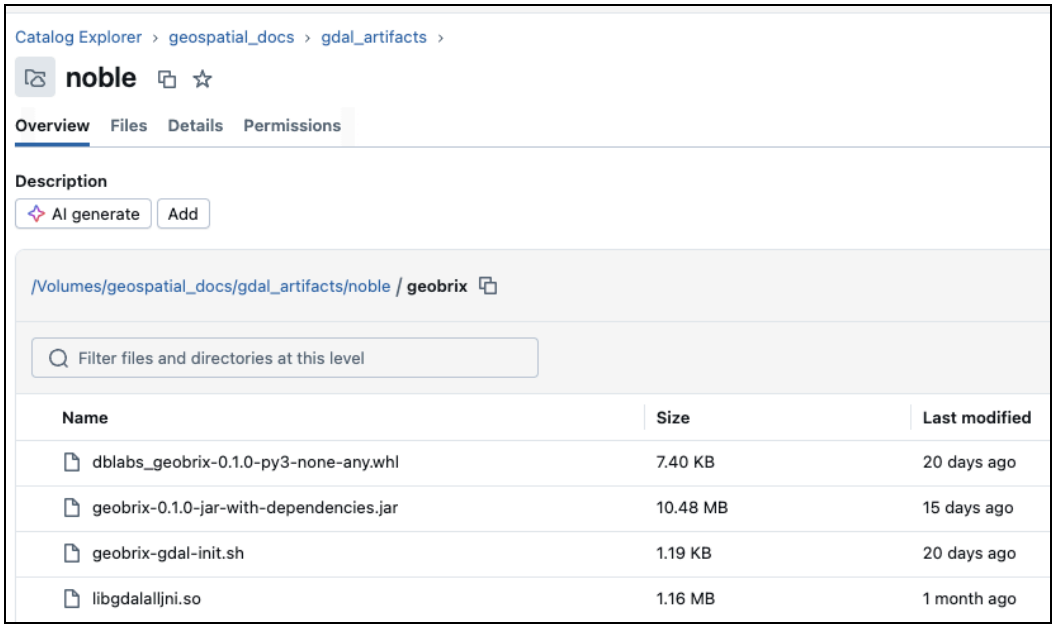
Second, while no customization is yet required as of OCT 2025, you will notice that the init script configures to get GDAL from UbuntuGIS PPA. Unfortunately, that apt repository does not pin different versions and is subject to update at any time. So, occasionally good to verify that the version is still 3.11.x. Since we are running with DBR 17.3 LTS, we will end up getting the “noble” package version (via the apt-get command).

If UbuntuGIS PPA advances to a version > 3.11, then you/we will have to rebuild JNI `.so` (shared object) to match. Note: The shared object is not for ARM machines, only x64!

[2] Add Artifacts to Volume

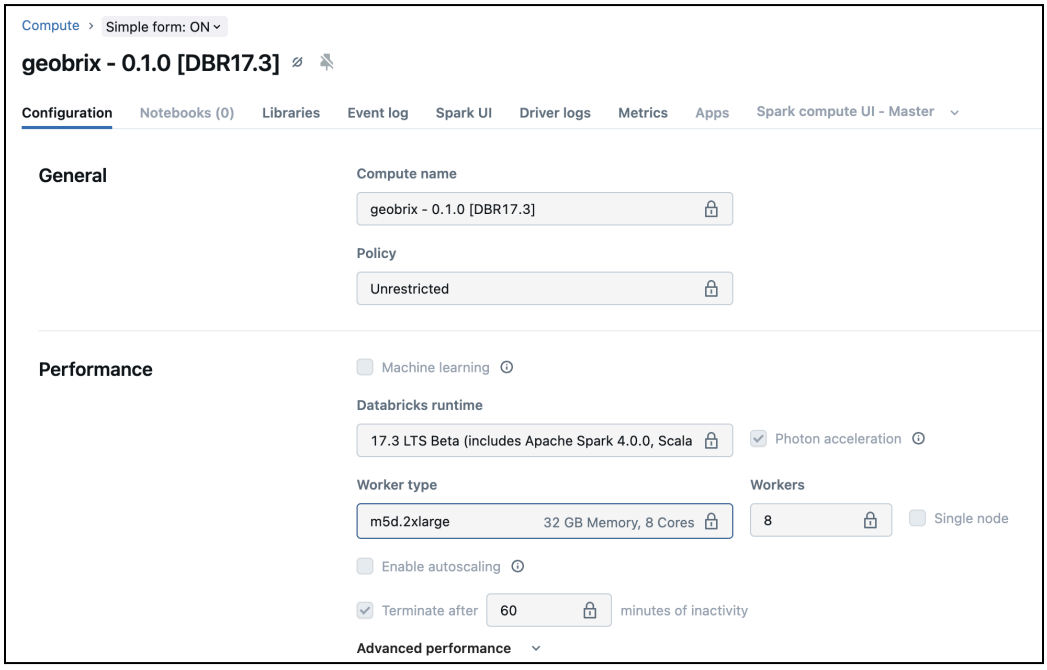
After configuring the init script, upload all artifacts to your preferred Volume location. Do not upload as a zip but rather the individual files. In this reference example we are within catalog 'geospatial_docs' and schema 'gdal_artifacts'.

We chose Volume 'noble' for the location of the artifacts and uploaded them accordingly.



[3] Cluster

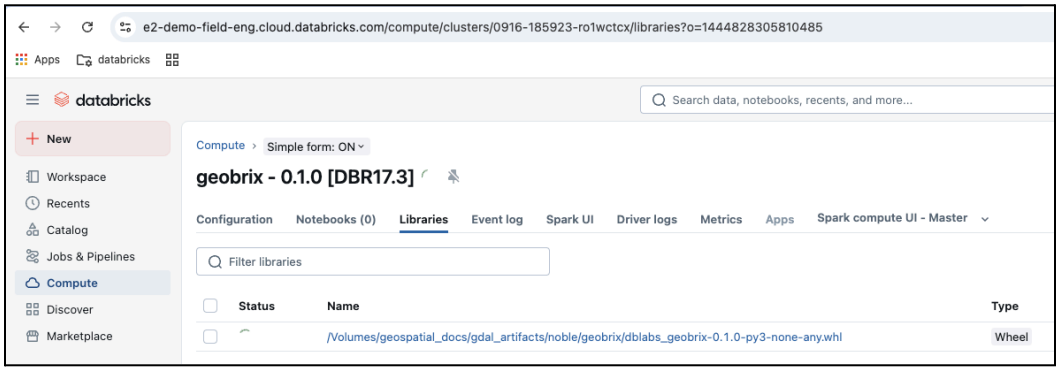
With the artifacts in the Volume, we can now configure our cluster. We need to choose DBR 17.3 LTS.



The Access mode needs to be Dedicated.

The Init Script is added from the Volume.

Then the WHL is also added from the Volume under the Cluster's Libraries Tab.



With that, you can hit 'Start' on the cluster!

Hello GeoBrix

Readers

The following spark readers are automatically registered with the JAR on the classpath.

Raster [“gdal”]

Options

- “sizeInMB” → defaults to “16” - split the file if over the threshold
- “filterRegex” → defaults to “.*” - filter loaded files from the provided directory

We are really only focused on GeoTiffs right now, but you are free to try to load any available driver with something like:

```
Python
(
  spark
    .read.format(“gdal”)
    .option("driverName", "GTiff") # if not provided, extension is used to
detect
    .load("<path>")
)
```

Table +

	source	tile
1	dbfs:/Volumes/geospatial_docs/gdal_artifacts/no...	{"cellid":-1,"raster":"SUkqAAgAAAAUAAABAwABAAAAYAkAAAEBAwABAAAAYAkA
2	dbfs:/Volumes/geospatial_docs/gdal_artifacts/no...	{"cellid":-1,"raster":"SUkqAAgAAAAUAAABAwABAAAAYAkAAAEBAwABAAAAYAkA
3	dbfs:/Volumes/geospatial_docs/gdal_artifacts/no...	{"cellid":-1,"raster":"SUkqAAgAAAAUAAABAwABAAAAYAkAAAEBAwABAAAAYAkA
4	dbfs:/Volumes/geospatial_docs/gdal_artifacts/no...	{"cellid":-1,"raster":"SUkqAAgAAAAUAAABAwABAAAAYAkAAAEBAwABAAAAYAkA
5	dbfs:/Volumes/geospatial_docs/gdal_artifacts/no...	{"cellid":-1,"raster":"SUkqAAgAAAAUAAABAwABAAAAYAkAAAEBAwABAAAAYAkA
6	dbfs:/Volumes/geospatial_docs/gdal_artifacts/no...	{"cellid":-1,"raster":"SUkqAAgAAAAUAAABAwABAAAAYAkAAAEBAwABAAAAYAkA
7	dbfs:/Volumes/geospatial_docs/gdal_artifacts/no...	{"cellid":-1,"raster":"SUkqAAgAAAAUAAABAwABAAAAYAkAAAEBAwABAAAAYAkA

Vector [“ogr”]

Options

- “driverName” → if not provided, GDAL uses best guess based on file extension
- “chunkSize” → default "10000" - number of records for multi-threading per file reading
- “layerN” → default “0” - for file formats that use layers
- “layerName” → default “” - for file formats that use layers
- “asWKB” → default “true” - whether to return WKB or WKT geometry results

Note: For the Beta, results are not converted to Databricks new native spatial types for GEOMETRY / GEOGRAPHY, so this would be an additional step once the data has been read.

Named Readers

The following are available and call the “ogr” reader with some options explicitly set.

Shapefile [“shapefile”]

This is a named OGR Reader, sets “driverName” → "ESRI Shapefile":

- GDAL auto-associates the following extensions to this driver: **.shz** files (ZIP files containing the .shp, .shx, .dbf and other side-car files of a single layer) and **.shp.zip** files (ZIP files containing one or several layers)
- With the named reader, GDAL can additionally handle **.zip** files (ZIP files containing one or several layers)

Python

```
(
  spark
    .read.format(“shapefile”)
    .load("<path_to_supported_files>")
)
```

The output will look something like the following, maintaining attribute columns and having 3 columns for geometry: ‘geom_0’, ‘geom_0_srid’, and ‘geom_0_srid_proj’.

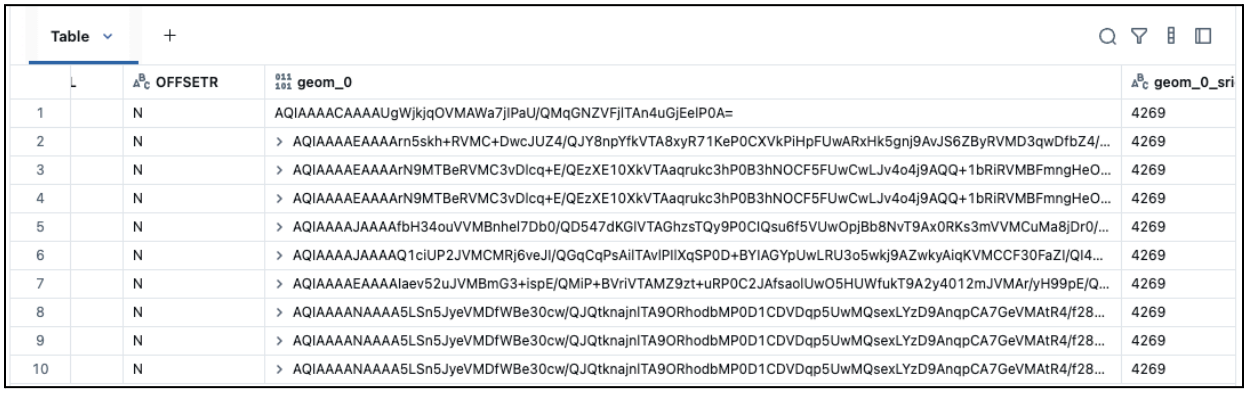


Table +

	L	OFFSETR	geom_0	geom_0_sri
1		N	AQIAAAACAAAAUgWjkjqOVMAWa7jlPaU/QMqGNZVFjlTAn4uGjEelP0A=	4269
2		N	› AQIAAAAEAAAArn5skh+RVMC+DwcJUZ4/QJY8npYfkVTA8xyR71KeP0CXVkPiHpFUwARxHk5gnj9AvJS6ZByRVMD3qwDfbZ4/...	4269
3		N	› AQIAAAAEAAAArN9MTBeRVMC3vDlcq+E/QEzXE10XkVTAaqrukc3hP0B3hNOCF5FUwCwLJv4o4j9AQQ+1bRiRVMBFmngHeO...	4269
4		N	› AQIAAAAEAAAArN9MTBeRVMC3vDlcq+E/QEzXE10XkVTAaqrukc3hP0B3hNOCF5FUwCwLJv4o4j9AQQ+1bRiRVMBFmngHeO...	4269
5		N	› AQIAAAAJAAAAfbH34ouVVMBnhel7Db0/QD547dKGlVTAGhzsTQy9P0ClQsu6f5VUwOpjBb8NvT9Ax0RKs3mVVMCuMa8jDr0/...	4269
6		N	› AQIAAAAJAAAAQ1ciUP2JVMCMRj6veJl/QGqCqPsAilTAvlPllXqSP0D+BYIAGYpUwLRU3o5wkj9AZwkyAiqKVMCCF30FaZl/QI4...	4269
7		N	› AQIAAAAEAAAAIaev52uJVMBmG3+ispE/QMiP+BVriVTAMZ9zt+uRP0C2JAfsaolUwO5HUWfukT9A2y4012mJVMAr/yH99pE/Q...	4269
8		N	› AQIAAAANAAAA5LSn5JyeVMDfWBe30cw/QJQtknajnlTA9ORhodbMP0D1CDVDqp5UwMQsexLYzD9AnqpCA7GeVMAtR4/f28...	4269
9		N	› AQIAAAANAAAA5LSn5JyeVMDfWBe30cw/QJQtknajnlTA9ORhodbMP0D1CDVDqp5UwMQsexLYzD9AnqpCA7GeVMAtR4/f28...	4269
10		N	› AQIAAAANAAAA5LSn5JyeVMDfWBe30cw/QJQtknajnlTA9ORhodbMP0D1CDVDqp5UwMQsexLYzD9AnqpCA7GeVMAtR4/f28...	4269

GeoJSON [“geojson”]

This is a named OGR Reader.

Options

- “multi” → default “true”
  - when “true” set “driverName” → "GeoJSONSeq"
  - otherwise | “GeoJSON”

```python
(
  spark
    .read.format("geojson")
      .option("multi", "false") # if not provided, "true" assumed
    .load("<path_to_supported_files>")
)
```

The output will look something like the following, maintaining attribute columns and having 3 columns for geometry: ‘geom_0’, ‘geom_0_srid’, and ‘geom_0_srid_proj’.

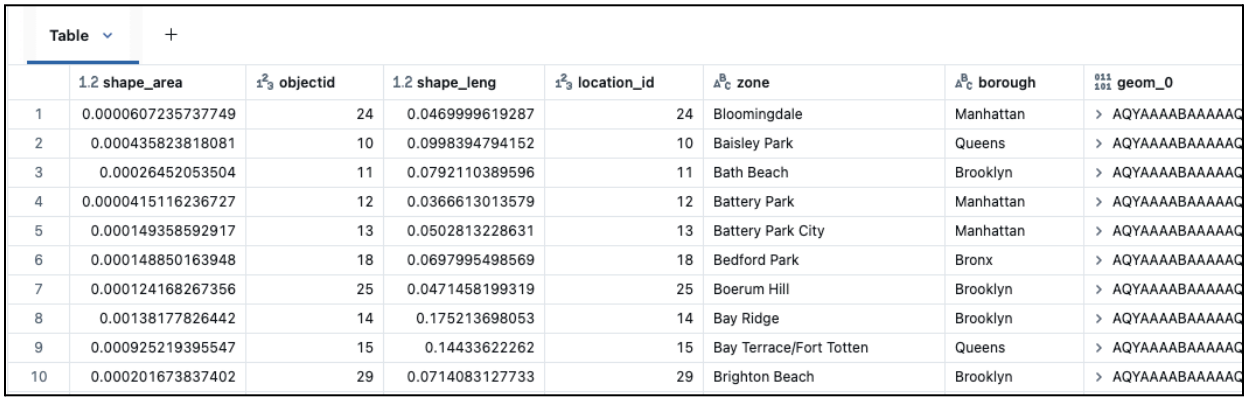


Table +

	1.2 shape_area	objectid	1.2 shape_leng	location_id	zone	borough	geom_0
1	0.0000607235737749	24	0.0469999619287	24	Bloomingdale	Manhattan	AQYAAAABAAAAAQ
2	0.000435823818081	10	0.0998394794152	10	Baisley Park	Queens	AQYAAAABAAAAAQ
3	0.00026452053504	11	0.0792110389596	11	Bath Beach	Brooklyn	AQYAAAABAAAAAQ
4	0.0000415116236727	12	0.0366613013579	12	Battery Park	Manhattan	AQYAAAABAAAAAQ
5	0.000149358592917	13	0.0502813228631	13	Battery Park City	Manhattan	AQYAAAABAAAAAQ
6	0.000148850163948	18	0.0697995498569	18	Bedford Park	Bronx	AQYAAAABAAAAAQ
7	0.000124168267356	25	0.0471458199319	25	Boerum Hill	Brooklyn	AQYAAAABAAAAAQ
8	0.00138177826442	14	0.175213698053	14	Bay Ridge	Brooklyn	AQYAAAABAAAAAQ
9	0.000925219395547	15	0.14433622262	15	Bay Terrace/Fort Totten	Queens	AQYAAAABAAAAAQ
10	0.000201673837402	29	0.0714083127733	29	Brighton Beach	Brooklyn	AQYAAAABAAAAAQ

GeoPackage [“gpkg”]

This is a named OGR Reader, sets “driverName” → "GPKG".

```python
(
  spark
    .read.format("ogr_gpkg")
    .load("<path_to_supported_files>")
)
```

The output will look something like the following, maintaining attribute columns and having 3 columns for geometry: ‘shape’, ‘shape_srid’, and ‘shape_srid_proj’.

Table +

	verified	collection_date	shape	shape_srid	shape_srid_proj
1	N	2023/05/19 12:03:59+00	AQEAAAAAeAsko7odQeqVsiTT9VNB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs
2	N	2024/07/29 11:24:01+00	AQEAAADAlpDPjnkSQfgx5kK/T1NB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs
3	N	2020/04/17 12:08:20+00	AQEAAACAc0ZUNzQeQbZif5kr/FJB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs
4	Y	2016/04/25 22:24:37+00	AQEAAACAUUn9EFshQaw+V6d/w1NB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs
5	N	2016/04/25 22:24:37+00	AQEAAADwp8bLK3QXQczuyWPsuFJB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs
6	N	2025/04/09 08:46:11+00	AQEAAADw0k0i1SAaQexRuEbAyVJB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs
7	Y	2015/09/27 17:20:24+00	AQEAAADgehSOrOsgQQg9m3W5u1NB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs
8	Y	2015/09/27 17:20:24+00	AQEAAAAYak1zZ+sgQewvu5vCu1NB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs
9	Y	2015/09/27 17:20:24+00	AQEAAACQdXHbh/MhQdiBczKFP1RB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs
10	N	2017/08/25 06:20:06+00	AQEAAABYn6ttuflgQbTqc739u1NB	26915	+proj=utm +zone=15 +datum=NAD83 +units=m +no_defs

File GeoDatabase [“file_gdb”]

This is a named OGR Reader, sets “driverName” → "OpenFileGDB". You also may want to specify options “layerN” or “layerName” to read the desired layer.

Python

```
(
  spark
    .read.format(“file_gdb”)
    .load("<path_to_supported_files>")
)
```

The output will look something like the following, maintaining attribute columns and having 3 columns for geometry: ‘SHAPE’, ‘SHAPE_srid’, and ‘SHAPE_srid_proj’. Note: column names are case insensitive.

Table +

	C_UNDER	CARRIED_LABEL	SHAPE	SHAPE_srid	SHAPE_srid_proj
1	0	COUNTY ROAD 9 :99'99"	AQEAAABAeVgoU48SQbivA4ca+VFB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs
2	0	FOREST BROOK ROAD :99'99"	AQEAAADAFyazmLwhQdrO948nXVFB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs
3	0	GRANDVIEW AVENUE :99'99"	AQEAAAD4Bl+4mJEhQQg9m+kUYVFB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs
4	0	WILFRED KING ROAD :99'99"	AQEAAAC4HoXr6DYiQV4py7y+51JB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs
5	0	CO RD 141 :99'99"	AQEAAABgMlVwnhlaQf5l90gRxFFB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs
6	0	COUNTY ROAD 116 :99'99"	AQEAAABwPQrXQWleQaYKRpHkmVFB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs
7	0	ENGLISH ROAD :99'99"	AQEAAABwGw2gYAkRQUi/femkRlJB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs
8	0	MILL ROAD :99'99"	AQEAAABQ/BgzCggRQbiNBhDZRVJB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs
9	0	COUNTY ROAD 253 :99'99"	AQEAAACAL0xm05ghQWlyVfhSClJB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs
10	0	CR 82 :99'99"	AQEAAADlEMcau1wiQUi/fWXgvlJB	26918	+proj=utm +zone=18 +datum=NAD83 +units=m +no_defs

Write to Lakehouse Tables

When you execute a reader, it translates (through the DataSourcev2 API) the as-provided data into a Spark DataFrame result. The natural next step, often inline to reading, would be to write

(or materialize) the results as a table. Writing is especially important for scaled data to drive better query performance.

Vector Data

Here is an example of writing a dataframe as-is to Lakehouse (geometries in WKB or WKT), showing “overwrite” mode. You might have other considerations on storing the data, including the catalog and schema location as well as converting to Databricks built-in spatial types which is shown next.

```python
(
 df
   .write
   .mode("overwrite")
   .option("overwriteSchema", "true")
   .saveAsTable("<tbl>")
)
```

Raster Data

For writing the actual raster data originating from the reader or other manipulations, use the “gdal” raster Data Source writer.

Structure

The schema of the table / dataframe provided to the the writer:

- “source” - optional column containing the initial source raster (limited uses)
- “tile” - Raster Tile as described in RasterX Functions

Options

- “ext” → default “tif” - specify if you have converted the extension to other than “tif”, e.g. via gbx_rst_asformat prior to writing

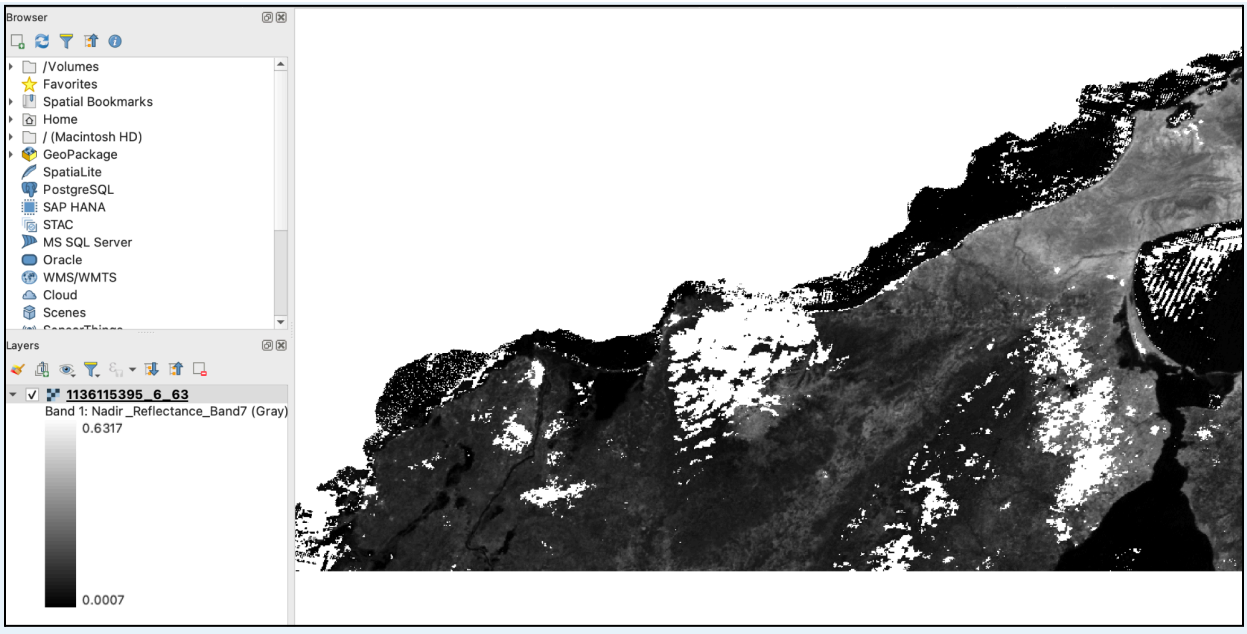
Additional

- `.mode(“append”)` - needs to be included in the write
- `.save(“<Volume”>)` - where to write the rasters; you may need to clean this location up for repeated runs of the same data

```python
(
```

```
    df.select("tile")       # not using "source" column
      .write.format(“gdal”)
      .mode("append")       # include "append" in the write
      .option("ext", "tif") # match to whatever the tile is ('tif' default)
      .save("<Volume>")     # volume dir to write files
  )
```

Here is an example of writing a single-band tif, loaded into QGIS.



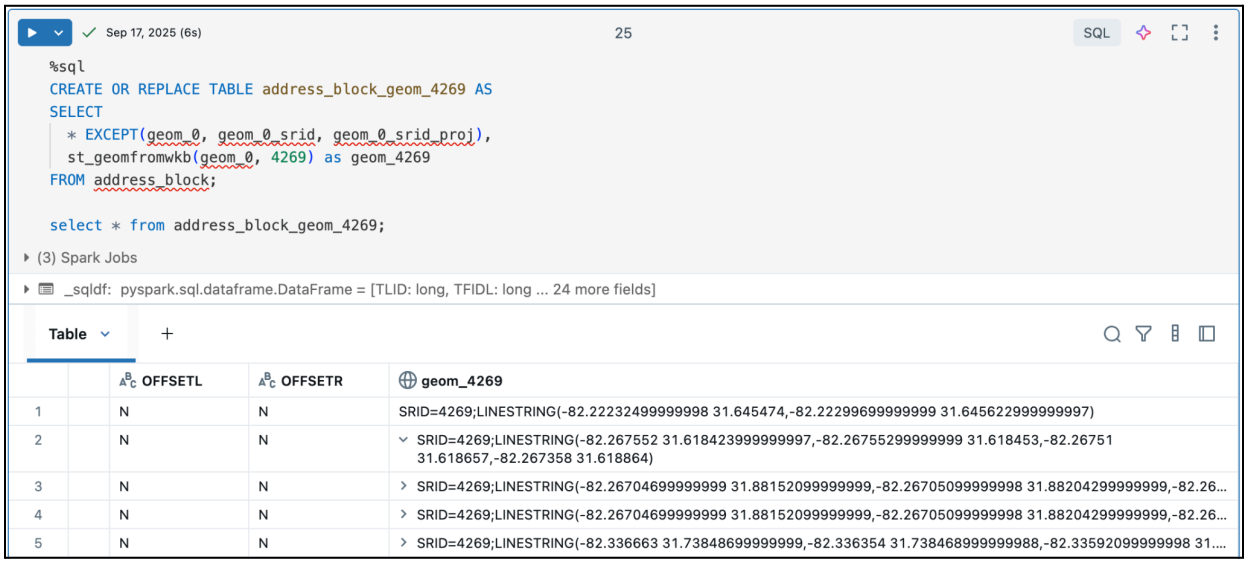
Using Databricks Native Types

Vector Data

Databricks supports converters to/from columnar WKB and WKT as well as GeoJSON for our native spatial types, see Import and Export Docs. In this instance we are focused on generating a table with GEOMETRY native type from the as-read format. Note: there is also growing support for GEOGRAPHY native types.

In the example below, we know that the data is in SRID EPSG:4269, which is North American Datum 1983, having units in degrees; a number of US Government datasets are standardized to NAD83. A very popular SRID is EPSG:4326, which is WGS84; it is used the most among all datasets. Databricks supports over 11K SRIDs, and has the ability to transform among them, see Spatial Reference System Functions.

The example uses Databricks native function ST_GeomFromWKB to convert WKB into our built-in GEOMETRY type.



Raster Data

Databricks does not currently offer a native raster type, so saving raster tables / dataframes would be either saving the Raster Tile as-is or potentially writing out resulting raster data to Volumes as shown with Write to Lakehouse Tables.

Here is a partial example using xView data. Notice how in this example we are including the bounding box, which is returned as a WKB. Note: It would be beneficial to use the conversion shown for Vector Data (just above) to go ahead and write a table with “bbox” column stored as a Databricks native GEOMETRY, e.g. using ST_GeomFromWKB(“bbox”, “srid”).

This example shows python bindings for RasterX Functions imported as `rx`. It also uses the function `rst_fromfile` (sql is gbx_rst_fromfile), an inline alternative to the “gdal” raster reader.

Python

```python
from pyspark.sql import functions as f
from databricks.labs.gbx.rasterx import functions as rx

# based on current catalog and schema in this example
if not spark.catalog.tableExists("xview_raster"):
 (
   tif_df
```

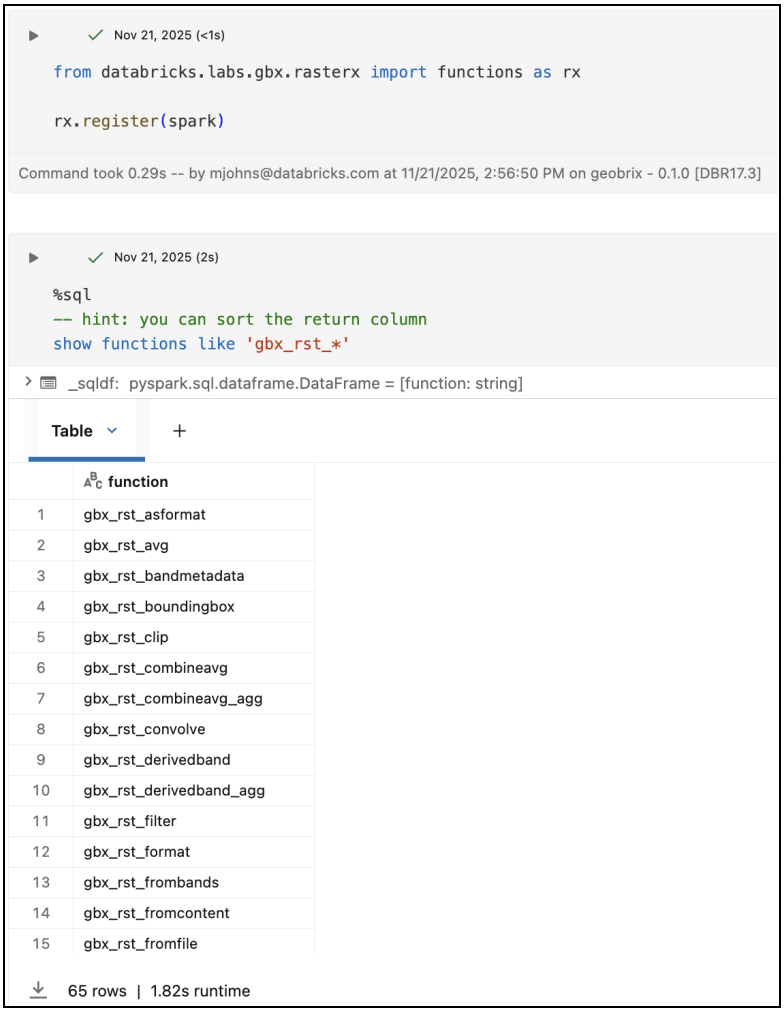
```
      .drop("content") # <- image binary
      .withColumn(
        "tile",
        rx.rst_fromfile("path", "GTiff")
      )
      .withColumn("bbox", rx.rst_boundingbox("tile"))
      .withColumn("srid", rx.rst_srid("tile"))
      .write
        .option("overwriteSchema", "true")
        .mode("overwrite")
        .saveAsTable("xview_raster")
  )

raster_df = spark.table("xview_raster")
raster_df.display()
```

Table +

	path	modificationTime	length	tile	bbox	srid
1	dbfs:/Volumes/us...	2025-08-04T00:39:45.000+00:00	32190176	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABcA3er...	4326
2	dbfs:/Volumes/us...	2025-08-04T00:39:41.000+00:00	27544946	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABUAkg...	4326
3	dbfs:/Volumes/us...	2025-08-04T00:39:42.000+00:00	27526670	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABUAkai...	4326
4	dbfs:/Volumes/us...	2025-08-04T00:39:42.000+00:00	27518531	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABUAkb...	4326
5	dbfs:/Volumes/us...	2025-08-04T00:39:43.000+00:00	27536804	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABUAkd...	4326
6	dbfs:/Volumes/us...	2025-08-04T00:39:43.000+00:00	27536804	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABUAke...	4326
7	dbfs:/Volumes/us...	2025-08-04T00:39:44.000+00:00	24230030	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABcBR6l...	4326
8	dbfs:/Volumes/us...	2025-08-04T00:39:44.000+00:00	24220724	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABcBR5...	4326
9	dbfs:/Volumes/us...	2025-08-04T00:39:45.000+00:00	24220724	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABcBR5...	4326
10	dbfs:/Volumes/us...	2025-08-04T00:39:46.000+00:00	32766947	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABb/4bd...	4326
11	dbfs:/Volumes/us...	2025-08-04T00:39:47.000+00:00	32766947	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABb/4SD...	4326
12	dbfs:/Volumes/us...	2025-08-04T00:39:51.000+00:00	23520488	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABcBYva...	4326
13	dbfs:/Volumes/us...	2025-08-04T00:39:51.000+00:00	24220724	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABcBR5...	4326
14	dbfs:/Volumes/us...	2025-08-04T00:39:52.000+00:00	27490112	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABUAkQ...	4326
15	dbfs:/Volumes/us...	2025-08-04T00:39:52.000+00:00	24230030	{"index_id":null,"raster":"SUkqAAgAA...	AIAAAAMAAAABAAAABcBR5...	4326

Language Bindings

Example import and registration of SQL functions; in this case showing RasterX Functions imported, then registered with Spark SQL.



Functions can be described with `describe function extended <fun_name>`. **Note: some text descriptions may not be complete for the Beta.** Additionally, functions are initially documented in the Appendix.

Here is an example of describing an individual function that has been registered:

```sql
describe function extended gbx_rst_boundingbox
```

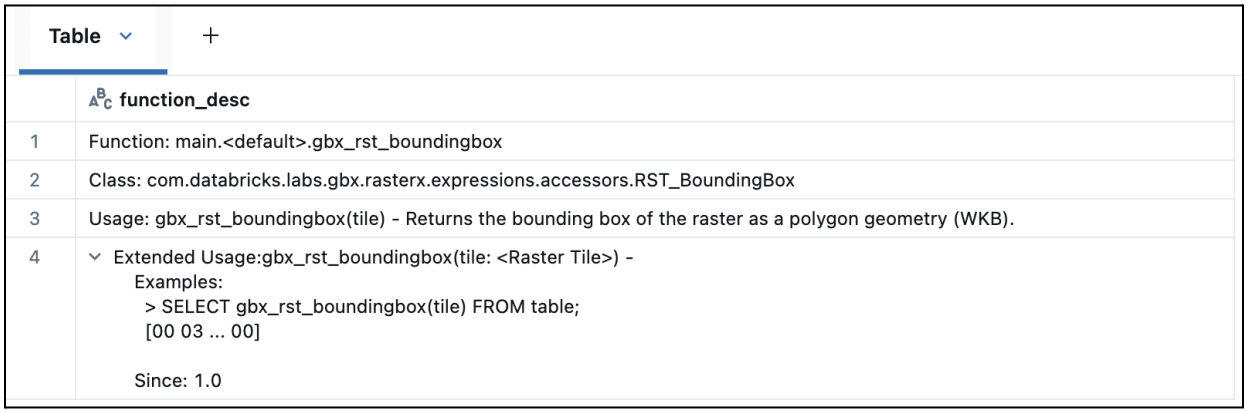


Table +

	function_desc
1	Function: main.<default>.gbx_rst_boundingbox
2	Class: com.databricks.labs.gbx.rasterx.expressions.accessors.RST_BoundingBox
3	Usage: gbx_rst_boundingbox(tile) - Returns the bounding box of the raster as a polygon geometry (WKB).
4	Extended Usage:gbx_rst_boundingbox(tile: <Raster Tile>) - Examples: > SELECT gbx_rst_boundingbox(tile) FROM table; [00 03 ... 00] Since: 1.0

Scala

The heavy-weight API is written in Scala with various spark optimizations implemented with best practices, including using Spark Connect to invoke the columnar expressions. The pattern for registering functions is `com.databricks.labs.gbx.<category>.functions` where 'gbx' is the convention for GeoBrix in classpaths:

- **VectorX** - sample function `vx.st_legacyaswkb.`
  - `import com.databricks.labs.gbx.vectorx.{functions => vx}`
  - `vx.register(spark)`
  - `vx.<function>`
- **GridX** (showing BNG) `import com.databricks.labs.gbx.gridx.bng.{functions => bx}` (same registrations and execution pattern as VectorX) - sample function `bx.bng_cellarea.`
- **RasterX** `import com.databricks.labs.gbx.rasterx.{functions => rx}` (same registrations and execution pattern as VectorX) - sample functions `rx.rst_clip`.

Python

The python bindings are a lightweight wrapper to the underlying Scala columnar expressions via Spark Connect. Functions are registered in a similar manner as with scala:

- **VectorX**
  - `from databricks.labs.gbx.vectorx import functions as vx`
  - `vx.register(spark)`
  - `vx.<function>`
- **GridX** (showing BNG) `from databricks.labs.gbx.gridx.bng import functions as bx` (same registrations and execution pattern as VectorX)
- **RasterX** `from databricks.labs.gbx.rasterx import functions as rx` (same registrations and execution pattern as VectorX)

SQL

All GeoBrix SQL functions will be registered with `gbx_` prefix. This reflects a lesson learned from previous experiences, where functions registered without a prefix is unattributable to any particular provider on classic compute, e.g. cannot tell whether `st_<function>` invoked within classic compute is from product or Mosaic or Sedona, etc., but usage will be easily attributable to GeoBrix when `gbx_st_<function>` is invoked:

- Sample vector expression: `gbx_st_legacyaswkb`
- Sample grid expression: `gbx_bng_cellarea`
- Sample raster expression: `gbx_rst_clip`

Appendix

The following is copied from provided notebook “GeoBrix - SQL Functions”

RasterX Functions

Showing python import with SQL registration and notional example per function (refer to previous section for scala import).

```python
from databricks.labs.gbx.rasterx import functions as rx

rx.register(spark)
```

All of the functions that have a `tile` param (referred to as ‘Raster Tile’) use an internal structure that is used for serialization / deserialization:

```
index_id: <cellid>
raster: <data>
    metadata:
            path: "..."
            last_error: ""
            all_parents: "/Volumes/..."
            driver: "GTiff"
            parentPath: "/Volumes/..."
            lastCommand: "..."
```

We are focused on GeoTiffs / COG file formats in the Beta.

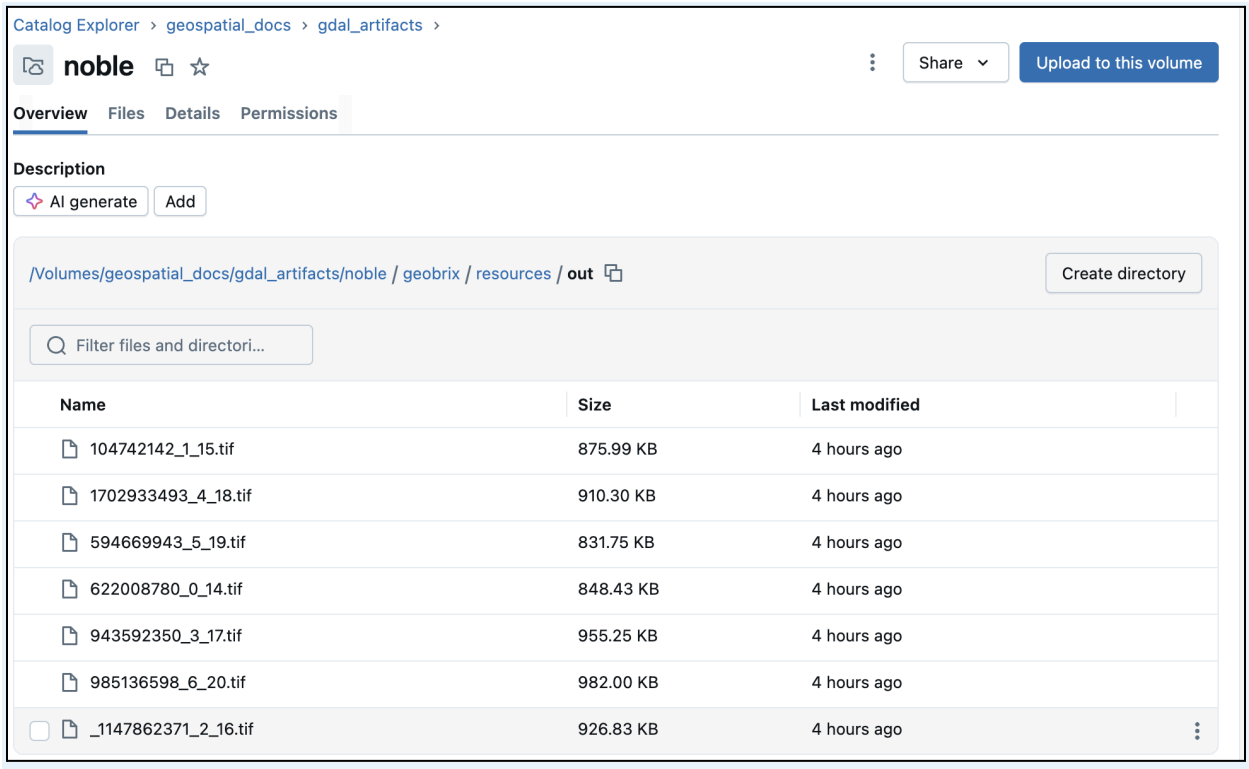
There are a few ways to get data from the as-provided files into the internal raster format used by GeoBrix, here are 2 to consider:

1. Use the “gdal” raster Data Source reader and specify “GTiff” as described in Raster [“gdal”].
2. For reading within a query there are functions such as gbx_rst_fromfile

For converting to another format, you *might* try gbx_rst_asformat; reminder functions are mostly focused on GTiff, so you could call this at the end of processing to convert to another format prior to writing the result out.

For writing, use the “gdal” raster Data Source writer as described in Raster Data.

```python
(
  spark
    .write.format(“gdal”)
    .mode("append")       # include "append" in the write
    .option("ext", "tif") # match to whatever the tile is ('tif' default)
    .save("<Volume>")     # volume dir for write
)
```



`gbx_rst_asformat(tile, new_format)`

Convert a tile to a new format.

Parameters:

- **tile** (*Column)* – Raster Tile
- **new_format** *(Column: StringType)* - short name of GDAL Raster Driver; not all may be available

Return type:

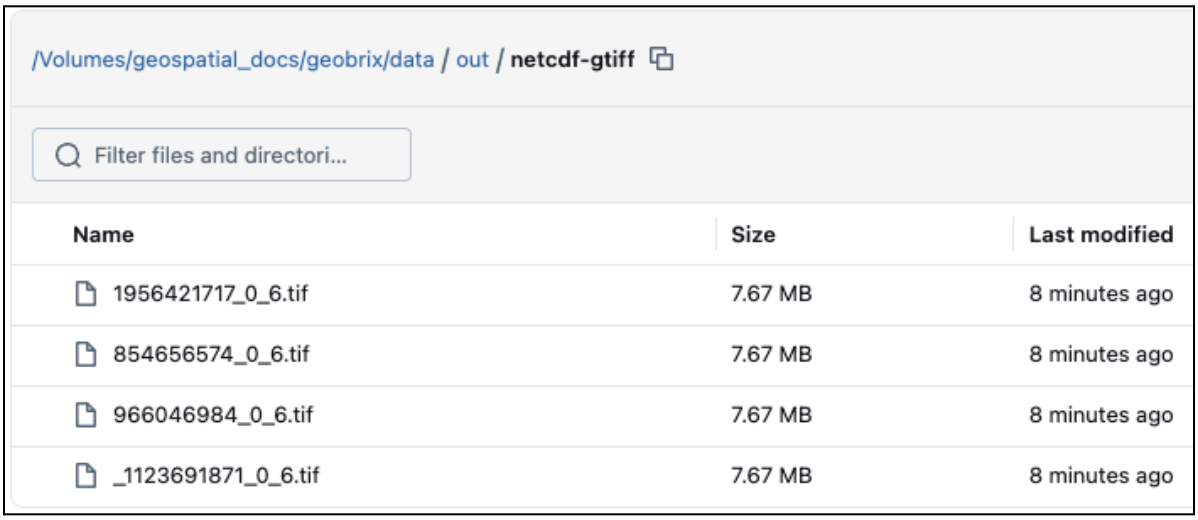
Column: BinaryType

Example:

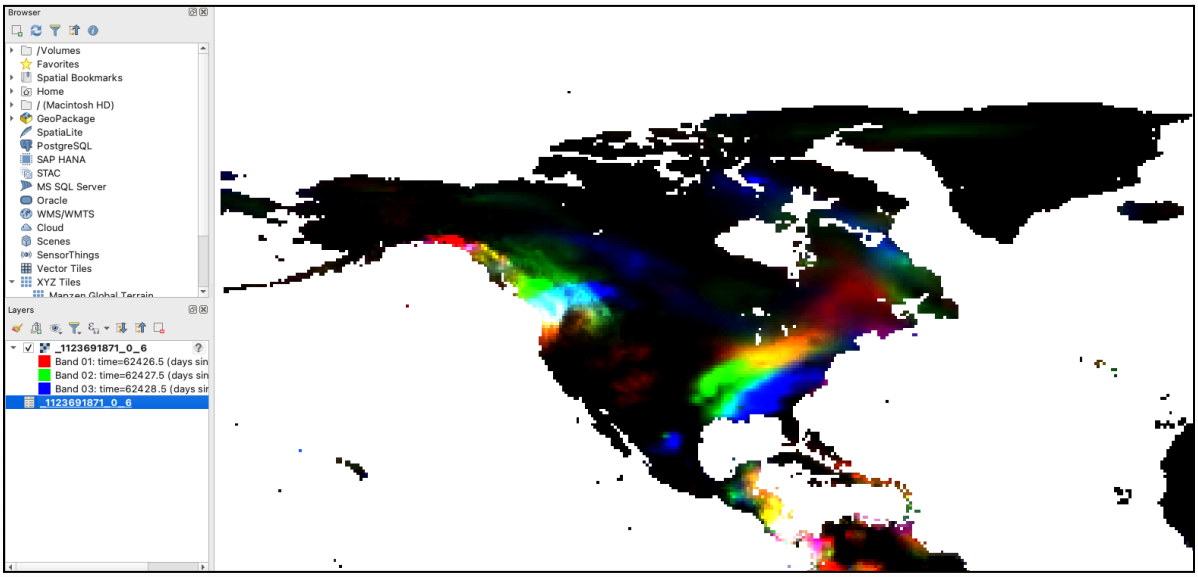
Showing a more complete example in Python for this, loading NetCDF and converting to GTiff. You may find that some formats are not configured or just don't work in the generalized way this library supports conversions.

```python
(
 spark.read.format("gdal")
   .option("driverName", "netCDF")
 .load("/Volumes/geospatial_docs/geobrix/data/netcdf/")
   .withColumn("tile", rx.rst_asformat("tile", f.lit("GTiff")))
 .write.format("gdal")
   .mode("append")       # include "append" in the write
   .option("ext", "tif") # 'tif' (default)
 .save("/Volumes/geospatial_docs/geobrix/data/out/netcdf-gtiff/")
)
```

Here is the NetCDF written to Volumes as GTiff.



Here is one of the resulting GTiff rasters (from NetCDF) loaded into QGIS.



`gbx_rst_avg(tile)`

Returns an array containing mean values for each band.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: ArrayType(DoubleType)

Example:

```
SQL
SELECT gbx_rst_avg(tile) FROM table LIMIT 1
```

```
+---------------------+
| gbx_rst_avg(tile) |
+---------------------+
| [42.0]              |
+---------------------+
```

`gbx_rst_bandmetadata(tile, band)`

Extract the metadata describing the raster band. Metadata is returned as a map of key value pairs.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **band** (*Column (IntegerType)*) – The band number from which to extract metadata.

Return type:

Column: MapType(StringType, StringType)

Example:

```
SQL
   SELECT gbx_rst_bandmetadata(tile, 1) FROM table LIMIT 1
```

```
+-------------------------------------------------------------------------------------+
| gbx_rst_bandmetadata(tile, 1)                                                       |
+-------------------------------------------------------------------------------------+
| {"_FillValue": "251", "NETCDF_DIM_time": "1294315200", "long_name": "bleaching alert |
| area 7-day maximum composite", "grid_mapping": "crs", "NETCDF_VARNAME":             |
| "bleaching_alert_area", "coverage_content_type": "thematicClassification",          |
| "standard_name": "N/A", "comment": "Bleaching Alert Area (BAA) values are coral     |
| bleaching heat stress levels: 0 - No Stress; 1 - Bleaching Watch; 2 - Bleaching     |
| Warning; 3 - Bleaching Alert Level 1; 4 - Bleaching Alert Level 2. Product          |
| description is provided at https://coralreefwatch.noaa.gov/product/5km/index.php.", |
| "valid_min": "0", "units": "stress_level", "valid_max": "4", "scale_factor": "1"}   |
+-------------------------------------------------------------------------------------+
```

`gbx_rst_boundingbox(tile)`

Returns the bounding box of the raster as a polygon geometry (WKB).

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: StructType(DoubleType, DoubleType, DoubleType, DoubleType)

Example:

```
SQL
SELECT gbx_rst_boundingbox(tile) FROM table LIMIT 1
```

```
+-------------------------------------------------------------------+
| gbx_rst_boundingbox(tile)                                         |
+-------------------------------------------------------------------+
| [00 03 ... 00] // WKB representation of the polygon bounding box |
+-------------------------------------------------------------------+
```

`gbx_rst_clip(tile, clip, cutline_all_touched)`

Clips `tile` using provided `clip` Geometry in a supported encoding (WKB, WKT).

Parameters:

- **tile** (*Column*) – Raster Tile.
- **clip** (*Column*) – Geometry (WKB | WKT) for raster clipping.
- **cutline_all_touched** (*Column (BooleanType)*) – A column to specify pixels boundary behavior.

Return type:

Column: Raster Tile

Notes

Geometry input

The `clip` parameter is expected to be a polygon or a multipolygon, can be WKB or WKT.

Cutline handling

The `cutline_all_touched` parameter:

- Optional: default is true. This is a GDAL warp config for the operation.
- If set to true, the pixels touching the geometry are included in the clip, regardless of half-in or half-out; this is important for tessellation behaviors.
- If set to false, only at least half-in pixels are included in the clip.
- More information can be found here

The actual GDAL command employed to perform the clipping operation is as follows:

`"gdalwarp -wo CUTLINE_ALL_TOUCHED=<TRUE|FALSE> -cutline <GEOMETRY> -crop_to_cutline"`

Output

Output raster tiles will have:

- the same extent as the input geometry.
- the same number of bands as the input raster.
- the same pixel data type as the input raster.
- the same pixel size as the input raster.
- the same coordinate reference system as the input raster.

Example:

```
SQL
SELECT gbx_rst_clip(tile, "POLYGON((0 0, 0 10, 10 10, 10 0, 0 0))") AS tile
FROM table LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_combineavg(tiles)`

Combines a collection of raster tiles by averaging the pixel values.

Parameters:

- **tiles** (*Column (ArrayType(Raster Tile))*) – Array of Raster Tiles.

Return type:

Column: Raster Tile

Notes

- Each tile in `tiles` must have the same extent, number of bands, pixel data type, pixel size and coordinate reference system.
- The output raster will have the same extent, number of bands, pixel data type, pixel size and coordinate reference system as the input tiles.

Example:

```
SQL
SELECT gbx_rst_combineavg(array(tile1,tile2,tile3)) AS tile FROM table LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_combineavg_agg(tile)`

Aggregates raster tiles by averaging pixel values.

Parameters:

- **tile** (*Column*) – A grouped column containing Raster Tiles.

Return type:

Column: Raster Tile

Notes

- Each `tile` must have the same extent, number of bands, pixel data type, pixel size and coordinate reference system.
- The output raster will have the same extent, number of bands, pixel data type, pixel size and coordinate reference system as the input tiles.

```
SQL
SELECT gbx_rst_combineavg_agg(tile) AS tile
FROM table
GROUP BY 1
LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_convolve(tile, kernel)`

Applies a convolution filter to the raster.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **kernel** (*Column (ArrayType(ArrayType(DoubleType)))*) – The kernel to apply to the raster.

Return type:

Column: Raster Tile

Notes

- The `kernel` can be Array of Array of either Double, Integer, or Decimal but will be cast to Double.
- This method assumes the kernel is square and has an odd number of rows and columns.
- Kernel uses the configured GDAL `blockSize` with a stride being `kernelSize/2`.

Example:

```
SQL
SELECT gbx_rst_convolve(tile, kernel) AS tile FROM table LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_derivedband(tile_expr, pyfunc, func_name)`

Combine tiles using the provided python function.

Parameters:

- **tile_expr** (*Column)* – Raster Tile.
- **pyfunc** (*Column (StringType)*) – A function to evaluate in python.
- **func_name** (*Column (StringType)*) – name of the function to evaluate in python.

Return type:

Column: Raster Tile

Notes

- Input raster tile(s) in `tile_expr` must have the same extent, number of bands, pixel data type, pixel size and coordinate reference system.
- The output raster will have the same the same extent, number of bands, pixel data type, pixel size and coordinate reference system as the input raster tile(s).

Example:

```
SQL
SELECT
gbx_rst_derivedband(array(tile1,tile2,tile3), py_func1, func1_name) AS tile
FROM SELECT (
```

```
date, tile1, tile2, tile3,
"""
import numpy as np
def average(in_ar, out_ar, xoff, yoff, xsize, ysize, raster_xsize,
raster_ysize, buf_radius, gt, **kwargs):
   out_ar[:] = np.sum(in_ar, axis=0) / len(in_ar)
""" as py_func1,
"average" as func1_name
FROM table
)
LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_derivedband_agg(tile, pyfunc, func_name)`

Combines a group by statement over aggregated raster tiles by using the provided python function.

Parameters:

- **tile** (*Column*) – A grouped column containing Raster Tile(s).
- **pyfunc** (*Column (StringType)*) – A function to evaluate in python.
- **func_name** (*Column (StringType)*) – name of the function to evaluate in python.

Return type:

Column: Raster Tile

Notes

- Input raster tiles in `tile` must have the same extent, number of bands, pixel data type, pixel size and coordinate reference system.
- The output raster will have the same the same extent, number of bands, pixel data type, pixel size and coordinate reference system as the input raster tiles.

Example:

```
SQL
SELECT
gbx_rst_derivedband_agg(tile, py_func1, func1_name) AS tile
FROM SELECT (
date, tile,
"""
import numpy as np
def average(in_ar, out_ar, xoff, yoff, xsize, ysize, raster_xsize,
raster_ysize, buf_radius, gt, **kwargs):
   out_ar[:] = np.sum(in_ar, axis=0) / len(in_ar)
""" as py_func1,
"average" as func1_name
FROM table
)
GROUP BY date, py_func1, func1_name
LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_filter(tile, kernel_size, operation)`

Applies a filter to the raster. Returns a new raster tile with the filter applied. `kernel_size` is the number of pixels to compare; it must be odd. `operation` is the op to apply, e.g. ‘avg’, ‘median’, ‘mode’, ‘max’, ‘min’.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **kernel_size** (*Column (IntegerType)*) – The size of the kernel. Has to be odd.
- **operation** (*Column (StringType)*) – The operation to apply to the kernel.

Return type:

Column: Raster Tile

Example:

```
SQL
SELECT gbx_rst_filter(tile, 3, "mode") AS tile FROM table LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_format(tile)`

Get the short name for the driver.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: StringType

Example:

```
SQL
SELECT gbx_rst_format(tile) FROM table LIMIT 1
```

```
+----------------------+
| gbx_rst_format(tile) |
+----------------------+
| GTiff                |
+----------------------+
```

`gbx_rst_frombands(bands)`

Combines a collection of raster tiles of different bands into a single raster.

Parameters:

- **bands** (*Column (ArrayType(Raster Tile))*) – Array of Raster Tiles.

Return type:

Column: Raster Tile

Notes

- All raster tiles must have the same extent.
- The tiles must have the same pixel coordinate reference system.
- The output tile will have the same extent as the input tiles.
- The output tile will have the a number of bands equivalent to the number of input tiles.
- The output tile will have the same pixel type as the input tiles.
- The output tile will have the same pixel size as the highest resolution input tile.
- The output tile will have the same coordinate reference system as the input tiles.

Example:

```
SQL
SELECT gbx_rst_frombands(array(tile1,tile2,tile3)) AS tile FROM table LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_fromcontent(content, driver)`

Returns a tile from raster data.

Parameters:

- **content** (*Column (BinaryType)*) – A column containing the raster data.
- **driver** (*Column(StringType)*) – GDAL driver to use to open the raster.

Return type:

Column: Raster Tile

The input raster must be a byte array in a BinaryType column.

Example:

```
SQL
CREATE TABLE IF NOT EXISTS TABLE tbl
     USING binaryFile
     OPTIONS (path "/Volumes/...");

SELECT gbx_rst_fromcontent(content) FROM tbl AS tile LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_fromfile(path, driver)`

Returns a raster tile from a file path.

Parameters:

- **path** (*Column (StringType)*) – Path to a raster file.
- **driver** (*Column (StringType)*) – GDAL driver short name.

Return type:

Column: Raster Tile

The file path must be a string.

Example:

```
SQL
CREATE TABLE IF NOT EXISTS TABLE tbl
     USING binaryFile
     OPTIONS (path "/Volumes/...");

SELECT gbx_rst_fromfile(path, 'GTiff') FROM tbl AS tile LIMIT 1
```

```
+--------------------------------------------------------------------------------+
| tile                                                                           |
+--------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
+--------------------------------------------------------------------------------+
```

`gbx_rst_georeference(tile)`

Returns GeoTransform of the raster tile as a Map. The output takes the form of a MapType with the following keys:

- "upperLeftX" → `geoTransform(0)`
- "upperLeftY" → `geoTransform(3)`
- "scaleX" → `geoTransform(1)`
- "scaleY" → `geoTransform(5)`
- "skewX" → `geoTransform(2)`
- "skewY" → `geoTransform(4)`

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: MapType(StringType, DoubleType)

Example:

```
SQL
SELECT gbx_rst_georeference(tile) FROM table LIMIT 1
```

```
+--------------------------------------------------------------------------------------------
---+
| gbx_rst_georeference(tile)
|
+--------------------------------------------------------------------------------------------
---+
| {"scaleY": -0.049999999152053956, "skewX": 0, "skewY": 0, "upperLeftY":
89.99999847369712, |
| "upperLeftX": -180.00000610436345, "scaleX": 0.050000001695656514}
|
+--------------------------------------------------------------------------------------------
---+
```

`gbx_rst_getnodata(tile)`

Returns the nodata value of the raster tile bands.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: ArrayType(DoubleType)

Example:

```
SQL
SELECT gbx_rst_getnodata(tile) FROM table LIMIT 1
```

```
+------------------------+
| gbx_rst_getnodata(tile) |
+------------------------+
| [0.0, -9999.0, ...]    |
+------------------------+
```

`gbx_rst_getsubdataset(tile, subset_name)`

Returns the subdataset of the raster tile with a given name.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **subset_name** (*Column (StringType)*) – Subdataset to return.

Return type:

Column: Raster Tile

Notes

- `name` should be the last identifier in the standard GDAL subdataset path: `DRIVER:PATH:NAME`.
- `subset_name` must be a valid subdataset name for the raster, i.e. it must exist within the raster.

Example:

```
SQL
SELECT gbx_rst_getsubdataset(tile, "sst") AS tile FROM table LIMIT 1
```

```
+--------------------------------------------------------------------------------
--------+
| tile
|
+--------------------------------------------------------------------------------
--------+
| {index_id: 593308294097928191, raster: [00 01 10 ... 00], parentPath: "...", driver:
"NetCDF" } |
+--------------------------------------------------------------------------------
--------+
```

gbx_rst_h3_rastertogridavg(tile, resolution)

Compute the gridwise mean of the pixel values in `tile`.

The result is a 2D array of cells, where each cell is a struct of (`cellID`, `value`).

Parameters:

- **tile** (*Column*) – Raster Tile.
- **resolution** (*Column (IntegerType)*) – H3 Resolution.

Return type:

Column: ArrayType(ArrayType(StructType(LongType, DoubleType)))

Notes

- To obtain cellID->value pairs, use the Spark SQL explode() function twice.
- The value/measure for each cell is the average of the pixel values in the cell.

Example:

```
SQL
SELECT gbx_rst_h3_rastertogridavg(tile, 3) FROM table
```

```
+--------------------------------------------------------------------------------
---------+
| gbx_rst_h3_rastertogridavg(tile, 3)
|
+--------------------------------------------------------------------------------
---------+
| [[{"cellID": "593176490141548543", "measure": 0}, {"cellID": "593386771740360703",
"measure":        |
| 1.2037735849056603}, {"cellID": "593308294097928191", "measure": 0}, {"cellID":
"593825202001936383" |
```

```
| , "measure": 0}, {"cellID": "593163914477305855", "measure": 2}, {"cellID":
"592998781574709247",    |
| "measure": 1.1283185840707965}, {"cellID": "593262526926422015", "measure": 2}, {"cellID":
|
| "592370479398911999", "measure": 0}, {"cellID": "593472602366803967", "measure":
0.3963963963963964} |
| , {"cellID": "593785619583336447", "measure": 0.6590909090909091}, {"cellID":
"591988330388783103",  |
| "measure": 1}, {"cellID": "592336738135834623", "measure": 1}, ....]]
|
+-----------------------------------------------------------------------------------------
---------+
```

`gbx_rst_h3_rastertogridcount(tile, resolution)`

Compute the gridwise count of the pixel values in `tile`.

The result is a 2D array of cells, where each cell is a struct of (`cellID`, `value`).

Parameters:

- **tile** (*Column*) – Raster Tile.
- **resolution** (*Column (IntegerType)*) – H3 Resolution.

Return type:

Column: ArrayType(ArrayType(StructType(LongType, IntegerType)))

Notes

- To obtain cellID->value pairs, use the Spark SQL explode() function twice.
- The value/measure for each cell is the count of the pixel values in the cell.

Example:

```
SQL
SELECT gbx_rst_h3_rastertogridcount(tile, 3) FROM table
```

```
+-----------------------------------------------------------------------------------------
---------+
| gbx_rst_h3_rastertogridcount(tile, 3)
|
+-----------------------------------------------------------------------------------------
---------+
| [[{"cellID": "593176490141548543", "measure": 1}, {"cellID": "593386771740360703",
"measure":        |
| 1}, {"cellID": "593308294097928191", "measure": 1}, {"cellID": "593825202001936383"
|
| , "measure": 1}, {"cellID": "593163914477305855", "measure": 2}, {"cellID":
"592998781574709247",     |
| "measure": 1}, {"cellID": "593262526926422015", "measure": 2}, {"cellID":
|
| "592370479398911999", "measure": 1}, {"cellID": "593472602366803967", "measure": 1}
|
| , {"cellID": "593785619583336447", "measure": 1}, {"cellID": "591988330388783103",
|
| "measure": 1}, {"cellID": "592336738135834623", "measure": 1}, ....]]
|
+-----------------------------------------------------------------------------------------
---------+
```

gbx_rst_h3_rastertogridmax(tile, resolution)

Compute the gridwise maximum of the pixel values in `tile`.

The result is a 2D array of cells, where each cell is a struct of (`cellID`, `value`).

Parameters:

- **tile** (*Column*) – Raster Tile.
- **resolution** (*Column (IntegerType)*) – H3 Resolution.

Return type:

Column: ArrayType(ArrayType(StructType(LongType, DoubleType)))

Notes

- To obtain cellID->value pairs, use the Spark SQL explode() function twice.
- The value/measure for each cell is the maximum of the pixel values in the cell.

Example:

```
SQL
SELECT gbx_rst_h3_rastertogridmax(tile, 3) FROM table
```

```
+-----------------------------------------------------------------------------------------
---------+
| gbx_rst_h3_rastertogridmax(tile, 3)
|
+-----------------------------------------------------------------------------------------
---------+
| [[{"cellID": "593176490141548543", "measure": 0}, {"cellID": "593386771740360703",
"measure":        |
| 1.2037735849056603}, {"cellID": "593308294097928191", "measure": 0}, {"cellID":
"593825202001936383" |
| , "measure": 0}, {"cellID": "593163914477305855", "measure": 2}, {"cellID":
"592998781574709247",    |
| "measure": 1.1283185840707965}, {"cellID": "593262526926422015", "measure": 2}, {"cellID":
|
| "592370479398911999", "measure": 0}, {"cellID": "593472602366803967", "measure":
0.3963963963963964} |
| , {"cellID": "593785619583336447", "measure": 0.6590909090909091}, {"cellID":
"591988330388783103",  |
| "measure": 1}, {"cellID": "592336738135834623", "measure": 1}, ....]]
|
+-----------------------------------------------------------------------------------------
---------+
```

gbx_rst_h3_rastertogridmedian(tile, resolution)

Compute the gridwise median of the pixel values in `tile`.

The result is a 2D array of cells, where each cell is a struct of (`cellID`, `value`).

Parameters:

- **tile** (*Column*) – Raster Tile.
- **resolution** (*Column (IntegerType)*) – H3 Resolution.

Return type:

Column: ArrayType(ArrayType(StructType(LongType, DoubleType)))

Notes

- To obtain cellID->value pairs, use the Spark SQL explode() function twice.
- The value/measure for each cell is the median of the pixel values in the cell.

Example:

```
SQL
SELECT gbx_rst_h3_rastertogridmedian(tile, 3) FROM table
```

```
+-----------------------------------------------------------------------------------------
---------+
| gbx_rst_h3_rastertogridmedian(tile, 3)
|
+-----------------------------------------------------------------------------------------
---------+
| [[{"cellID": "593176490141548543", "measure": 0}, {"cellID": "593386771740360703",
"measure":        |
| 1.2037735849056603}, {"cellID": "593308294097928191", "measure": 0}, {"cellID":
"593825202001936383" |
| , "measure": 0}, {"cellID": "593163914477305855", "measure": 2}, {"cellID":
"592998781574709247",    |
| "measure": 1.1283185840707965}, {"cellID": "593262526926422015", "measure": 2}, {"cellID":
|
| "592370479398911999", "measure": 0}, {"cellID": "593472602366803967", "measure":
0.3963963963963964} |
| , {"cellID": "593785619583336447", "measure": 0.6590909090909091}, {"cellID":
"591988330388783103",  |
| "measure": 1}, {"cellID": "592336738135834623", "measure": 1}, ....]]
|
+-----------------------------------------------------------------------------------------
---------+
```

`gbx_rst_h3_rastertogridmin(tile, resolution)`

Compute the gridwise minimum of the pixel values in `tile`.

The result is a 2D array of cells, where each cell is a struct of (`cellID`, `value`).

Parameters:

- **tile** (*Column*) – Raster Tile.
- **resolution** (*Column (IntegerType)*) – H3 Resolution.

Return type:

Column: ArrayType(ArrayType(StructType(LongType|StringType, DoubleType)))

Notes

- To obtain cellID->value pairs, use the Spark SQL explode() function twice.
- The value/measure for each cell is the minimum of the pixel values in the cell.

Example:

```
SQL
SELECT gbx_rst_h3_rastertogridmin(tile, 3) FROM table
```

```
+-----------------------------------------------------------------------------------------
---------+
| gbx_rst_h3_rastertogridmin(tile, 3)
|
+-----------------------------------------------------------------------------------------
---------+
| [[{"cellID": "593176490141548543", "measure": 0}, {"cellID": "593386771740360703",
"measure":        |
| 1.2037735849056603}, {"cellID": "593308294097928191", "measure": 0}, {"cellID":
"593825202001936383" |
| , "measure": 0}, {"cellID": "593163914477305855", "measure": 2}, {"cellID":
"592998781574709247",    |
| "measure": 1.1283185840707965}, {"cellID": "593262526926422015", "measure": 2}, {"cellID":
|
| "592370479398911999", "measure": 0}, {"cellID": "593472602366803967", "measure":
0.3963963963963964} |
| , {"cellID": "593785619583336447", "measure": 0.6590909090909091}, {"cellID":
"591988330388783103",  |
| "measure": 1}, {"cellID": "592336738135834623", "measure": 1}, ....]]
|
+-----------------------------------------------------------------------------------------
---------+
```

`gbx_rst_h3_tessellate(tile, resolution)`

Divides the raster tile into tessellating chips for the given resolution of the supported grid (H3, BNG, Custom). The result is a collection of new raster tiles.

Each tile in the tile set corresponds to an index cell intersecting the bounding box of the `tile`.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **resolution** (*Column (IntegerType)*) – H3 Resolution.

Return type:

Column: Raster Tile

Notes

- The result set is automatically exploded into a row-per-index-cell.
- If rst_merge is called on the output tile set, the original raster will be reconstructed.
- Each output tile chip will have the same number of bands as its parent `tile`.

Example:

```
SQL
SELECT gbx_rst_h3_tessellate(tile, 10) AS tile FROM table
```

```
+-----------------------------------------------------------------------------------------------
---+
| tile
|
+-----------------------------------------------------------------------------------------------
---+
| {index_id: 593308294097928191, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff"
} |
| {index_id: 593308294097928192, raster: [00 03 10 ... 00], parentPath: "...", driver: "GTiff"
} |
+-----------------------------------------------------------------------------------------------
---+
```

`gbx_rst_height(tile)`

Returns the height of the raster tile in pixels.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: IntegerType

Example:

```
SQL
SELECT gbx_rst_height(tile) FROM table
```

```
+-------------------------+
| gbx_rst_height(tile)    |
+-------------------------+
| 3600                    |
| 3600                    |
+-------------------------+
```

`gbx_rst_initnodata(tile)`

Initializes the nodata value of the raster tile bands based on their data type.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: Raster Tile

Notes

- The nodata value will be set to a default sentinel values according to the pixel data type of the raster bands.
- The output raster will have the same extent as the input raster.

Data Type	Scala representation	Value
ByteType		0
UnsignedShortType	`UShort.MaxValue`	65535
ShortType	`Short.MinValue`	-32768
UnsignedIntegerType	`Int.MaxValue`	4.294967294E9
IntegerType	`Int.MinValue`	-2147483648
FloatType	`Float.MinValue`	-3.4028234663852886E38
DoubleType	`Double.MinValue`	

Example:

```
SQL
SELECT gbx_rst_initnodata(tile) AS tile FROM table LIMIT 1
```

```
+-----------------------------------------------------------------------------------------
---+
| tile
|
+-----------------------------------------------------------------------------------------
---+
| {index_id: 593308294097928191, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff"
} |
+-----------------------------------------------------------------------------------------
---+
```

gbx_rst_isempty(tile)

Returns true if the raster tile is empty.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: BooleanType

Example:

```
SQL
SELECT gbx_rst_isempty(tile) FROM table
```

```
+---------------------+
|gbx_rst_isempty(tile) |
+---------------------+
|false                |
|false                |
+---------------------+
```

gbx_rst_maketiles(tile, size_in_mb)

Subdivide the raster into tiles of the given size in MB.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **size_in_mb** (*Column(IntegerType)*) – The size of the tiles in MB.

Return type:

Column: Raster Tile

Notes:

- If `size_in_mb` is set to -1, the file is loaded and returned as a single tile
- If set to 0, the file is loaded and subdivided into tiles of size 64MB
- If set to a positive value, the file is loaded and subdivided into tiles of the specified size
- If the file is too big to fit in memory, it is subdivided into tiles of size 64MB.

Example:

```
SQL
SELECT gbx_rst_maketiles(tile, 16) AS tile FROM table LIMIT 1
```

```
+-----------------------------------------------------------------------------------------
---+
| tile
|
+-----------------------------------------------------------------------------------------
---+
| {index_id: 593308294097928191, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff"
} |
+-----------------------------------------------------------------------------------------
---+
```

`gbx_rst_mapalgebra(tiles, expression)`

Performs map algebra on the raster tile.

Employs the `gdal_calc` command line raster calculator with standard numpy syntax.

Use any basic arithmetic supported by numpy arrays (such as +, -, *, and /) along with logical operators (such as >, <, =).

For this distributed implementation, all rasters must have the same dimensions and no projection checking is performed.

The `json_spec` parameter

- Input rasters to the algebra function are referenceable as variables with names `A` through `Z`.
- Bands from the input `tile` are referencable using ordinal 0..n values.

Examples of valid `json_spec`

```
(1) '{"calc": "A+B/C"}'
(2) '{"calc": "A+B/C", "A_index": 0, "B_index": 1, "C_index": 1}'
(3) '{"calc": "A+B/C", "A_index": 0, "B_index": 1, "C_index": 2, "A_band": 1, "B_band": 1,
"C_band": 1}'
```

In these examples:

1. demonstrates default indexing (i.e. the first three bands in `tile` are assigned A, B and C respectively)
2. demonstrates reusing an index (B and C represent the same band); and
3. shows band indexing.

Parameters:

- **tiles** (*Column*) – ArrayType[Raster Tile].
- **expression** (*Column (StringType)*) – A column containing the map algebra operation specification.

Return type:

Column: Raster Tile

Example:

```
SQL
   SELECT gbx_rst_mapalgebra(tile, "{calc: 'A+B', A_index: 0, B_index: 1}") as
   tile FROM table LIMIT 1
```

```
+-------------------------------------------------------------------------------------------
-------+
| tile
|
+-------------------------------------------------------------------------------------------
-------+
```

```
| {index_id: 593308294097928191, raster: [00 01 10 ... 00], parentPath: "...", driver:
"GTiff" } |
+-----------------------------------------------------------------------------------------
-------+
```

`gbx_rst_max(tile)`

Returns an array containing maximum values for each band.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: ArrayType(DoubleType)

Example:

```
SQL
SELECT gbx_rst_max(tile) FROM table LIMIT 1
```

```
+----------------------+
| gbx_rst_max(tile) |
+----------------------+
| [42.0]               |
+----------------------+
```

`gbx_rst_median(tile)`

Returns an array containing median values for each band.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: ArrayType(DoubleType)

Example:

```
SQL
SELECT gbx_rst_median(tile) FROM table LIMIT 1
```

```
+----------------------+
| gbx_rst_median(tile) |
+----------------------+
| [42.0]               |
```

```
+----------------------+
```

`gbx_rst_memsize(tile)`

Returns size of the raster tile in bytes.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: LongType

Example:

```
SQL
SELECT gbx_rst_memsize(tile) FROM table
```

```
+----------------------+
| gbx_rst_memsize(tile) |
+----------------------+
| 730260               |
| 730260               |
+----------------------+
```

`gbx_rst_merge(tiles)`

Combines a collection of raster tiles into a single raster.

Parameters:

- **tiles** (*Column*) – Array[Raster Tile].

Return type:

Column: Raster Tile

Example:

```
SQL
   SELECT gbx_rst_merge(array(tile1,tile2,tile3)) FROM table LIMIT 1
```

```
+-------------------------------------------------------------------------------------------
-------+
| gbx_rst_merge(array(tile1,tile2,tile3))
|
+-------------------------------------------------------------------------------------------
-------+
| {index_id: 593308294097928191, raster: [00 01 10 ... 00], parentPath: "...", driver:
"GTiff" } |
```

```
+-------------------------------------------------------------------------------------
-------+
```

`gbx_rst_merge_agg(tile)`

Aggregates raster tiles into a single raster.

Parameters:

- **tile** (*Column*) – A grouped column containing Raster Tile(s).

Return type:

Column: Raster Tile

Example:

```
SQL
    SELECT gbx_rst_merge_agg(tile)
    FROM table
    GROUP BY date
```

```
+-------------------------------------------------------------------------------------
-------+
| gbx_rst_merge_agg(tile)                                                            |
+-------------------------------------------------------------------------------------
-------+
| {index_id: 593308294097928191, raster: [00 01 10 ... 00], parentPath: "...", driver:
"GTiff" } |
+-------------------------------------------------------------------------------------
-------+
```

`gbx_rst_metadata(tile)`

Extract the metadata describing the raster tile. Metadata is return as a map of key value pairs.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: MapType(StringType, StringType)

Example:

```
SQL
SELECT gbx_rst_metadata(tile) FROM table LIMIT 1
```

```
+--------------------------------------------------------------------------------
---------+
| gbx_rst_metadata(tile)
|
+--------------------------------------------------------------------------------
---------+
| {"NC_GLOBAL#publisher_url": "https://coralreefwatch.noaa.gov",
"NC_GLOBAL#geospatial_lat_units": |
| "Degrees_north", "NC_GLOBAL#platform_vocabulary": "NOAA NODC Ocean Archive System
Platforms",    |
| "NC_GLOBAL#creator_type": "group", "NC_GLOBAL#geospatial_lon_units": "degrees_east",
|
| "NC_GLOBAL#geospatial_bounds": "POLYGON((-90.0 180.0, 90.0 180.0, 90.0 -180.0, -90.0
-180.0,     |
| -90.0 180.0))", "NC_GLOBAL#keywords": "Oceans > Ocean Temperature > Sea Surface
Temperature,     |
| Oceans > Ocean Temperature > Water Temperature, Spectral/Engineering > Infrared
Wavelengths >    |
| Thermal Infrared, Oceans > Ocean Temperature > Bleaching Alert Area",
"NC_GLOBAL#geospatial_lat_ | | max": "89.974998", .... (truncated)....
"NC_GLOBAL#history": "This is a product data file of the |
| NOAA Coral Reef Watch Daily Global 5km Satellite Coral Bleaching Heat Stress Monitoring
Product  |
| Suite Version 3.1 (v3.1) in its NetCDF Version 1.0 (v1.0).",
"NC_GLOBAL#publisher_institution":  |
| "NOAA/NESDIS/STAR Coral Reef Watch Program", "NC_GLOBAL#cdm_data_type": "Grid"}
|
+--------------------------------------------------------------------------------
---------+
```

gbx_rst_min(tile)

Returns an array containing minimum values for each band.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: ArrayType(DoubleType)

Example:

```
SQL
   SELECT gbx_rst_min(tile) FROM table LIMIT 1
```

```
+------------------+
| gbx_rst_min(tile) |
+------------------+
| [42.0]           |
```

```
+------------------+
```

gbx_rst_ndvi(tile, red_band, nir_band)

Calculates the Normalized Difference Vegetation Index (NDVI) for a raster.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **red_band** (*Column (IntegerType)*) – A column containing the band number of the red band.
- **nir_band** (*Column (IntegerType)*) – A column containing the band number of the near infrared band.

Return type:

Column: Raster Tile

NDVI is calculated using the formula: (NIR - RED) / (NIR + RED).

The output raster tiles will have:

- the same extent as the input raster.
- a single band.
- a pixel data type of float64.
- the same coordinate reference system as the input raster.

Example:

```
SQL
SELECT gbx_rst_ndvi(tile, 1, 2) FROM table LIMIT 1
```

```
+-----------------------------------------------------------------------------------------------
---+
| gbx_rst_ndvi(tile, 1, 2)
|
+-----------------------------------------------------------------------------------------------
---+
| {index_id: 593308294097928191, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff"
} |
+-----------------------------------------------------------------------------------------------
---+
```

`gbx_rst_numbands(tile)`

Returns number of bands in the raster tile.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: IntegerType

Example:

```
SQL
SELECT gbx_rst_numbands(tile) FROM table
```

```
+------------------------+
| gbx_rst_numbands(tile) |
+------------------------+
| 1                      |
| 1                      |
+------------------------+
```

`gbx_rst_pixelcount(tile)`

Returns an array containing valid pixel count values for each band.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: ArrayType(LongType)

Example:

```
SQL
   SELECT gbx_rst_pixelcount(tile) FROM table
```

```
+--------------------------+
| gbx_rst_pixelcount(tile) |
+--------------------------+
| [120560172]              |
+--------------------------+
```

`gbx_rst_pixelheight(tile)`

Returns the height of the pixel in the raster tile derived via GeoTransform.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: DoubleType

Example:

```
SQL
SELECT gbx_rst_pixelheight(tile) FROM table
```

```
+-------------------------+
| gbx_rst_pixelheight(tile) |
+-------------------------+
| 1                       |
| 1                       |
+-------------------------+
```

`gbx_rst_pixelwidth(tile)`

Returns the width of the pixel in the raster tile derived via GeoTransform.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: DoubleType

Example:

```
SQL
SELECT gbx_rst_pixelwidth(tile) FROM table
```

```
+-------------------------+
| gbx_rst_pixelwidth(tile) |
+-------------------------+
| 1                       |
| 1                       |
+-------------------------+
```

`gbx_rst_rastertoworldcoord(tile, pixel_x, pixel_y)`

Computes the world coordinates of the raster tile at the given x and y pixel coordinates.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **pixel_x** (*Column (IntegerType)*) – x coordinate of the pixel.
- **pixel_y** (*Column (IntegerType)*) – y coordinate of the pixel.

Return type:

Column: ArrayType[DoubleType]

Notes

- The result is a WKT point geometry.
- The coordinates are computed using the GeoTransform of the raster to respect the projection.

Example:

```
SQL
    SELECT gbx_rst_rastertoworldcoordy(tile, 3, 3) FROM table
```

```
+-----------------------------------------------+
| gbx_rst_rastertoworldcoordy(tile, 3, 3)       |
+-----------------------------------------------+
| POINT (-179.85000609927647 89.84999847624096) |
+-----------------------------------------------+
```

`gbx_rst_rastertoworldcoordx(tile pixel_x, pixel_y)`

Computes the world coordinates of the raster tile at the given x and y pixel coordinates.

The result is the X coordinate of the point after applying the GeoTransform of the raster.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **pixel_x** (*Column (IntegerType)*) – x coordinate of the pixel.

- **pixel_y** (*Column (IntegerType)*) – y coordinate of the pixel.

Return type:
Column: DoubleType

Example:

```
SQL
   SELECT gbx_rst_rastertoworldcoordx(tile, 3, 3) FROM table
```

```
+----------------------------------------+
| gbx_rst_rastertoworldcoordx(tile, 3, 3) |
+----------------------------------------+
| -179.85000609927647                    |
+----------------------------------------+
```

gbx_rst_rastertoworldcoordy(tile pixel_x, pixel_y)

Computes the world coordinates of the raster tile at the given x and y pixel coordinates.

The result is the Y coordinate of the point after applying the GeoTransform of the raster.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **pixel_x** (*Column (IntegerType)*) – x coordinate of the pixel.
- **pixel_y** (*Column (IntegerType)*) – y coordinate of the pixel.

Return type:
Column: DoubleType

Example:

```
SQL
   SELECT gbx_rst_rastertoworldcoordy(tile, 3, 3) FROM table
```

```
+----------------------------------------+
| gbx_rst_rastertoworldcoordy(tile, 3, 3) |
+----------------------------------------+
| 89.84999847624096                      |
+----------------------------------------+
```

`gbx_rst_retile(tile, tile_width, tile_height)`

Retiles the raster tile to the given size. The result is a collection of new raster tiles.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **tile_width** (*Column (IntegerType)*) – The width of the tiles.
- **tile_height** (*Column (IntegerType)*) – The height of the tiles.

Return type:

Column: Raster Tile

Example:

```
SQL
SELECT gbx_rst_retile(tile, 300, 300) FROM table
```

```
+-------------------------------------------------------------------------------------------
---------+
| gbx_rst_retile(tile, 300, 300)
|
+-------------------------------------------------------------------------------------------
---------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" }
|
| {index_id: …, raster: [00 03 10 ... 00], parentPath: "...", driver: "GTiff" }
|
+-------------------------------------------------------------------------------------------
---------+
```

`gbx_rst_rotation(tile)`

Computes the angle of rotation between the X axis of the raster tile and geographic North in degrees using the GeoTransform of the raster.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: DoubleType

Example:

```
SQL
   SELECT gbx_rst_rotation(tile) FROM table
```

```
+------------------------+
| gbx_rst_rotation(tile) |
+------------------------+
| 1.2                    |
| 21.2                   |
+------------------------+
```

gbx_rst_scalex(tile)

Computes the scale of the raster tile in the X direction.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: DoubleType

Example:

```
SQL
   SELECT gbx_rst_scalex(tile) FROM table
```

```
+----------------------+
| gbx_rst_scalex(tile) |
+----------------------+
| 1.2                  |
+----------------------+
```

gbx_rst_scaley(tile)

Computes the scale of the raster tile in the Y direction.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: DoubleType

Example:

```
SQL
   SELECT gbx_rst_scaley(tile) FROM table
```

```
+----------------------+
| gbx_rst_scaley(tile) |
+----------------------+
| 1.2                  |
```

```
+---------------------+
```

`gbx_rst_separatebands(tile)`

Returns a set of new single-band rasters, one for each band in the input raster. The result set will contain one row per input band for each `tile` provided and will add field ‘bandIndex’.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: Raster Tile

⚠️ Before performing this operation, you may want to add an identifier column to the dataframe to trace each band back to its original parent raster.

Example:

```
SQL
SELECT rst_separatebands(tile) FROM table
```

```
+-----------------------------------------------------------------------------------------
---------+
| tile
|
+-----------------------------------------------------------------------------------------
---------+
| {"index_id":null,"raster":"SUkqAAg...= (truncated)",
|
|
"metadata":{"path":"....tif","last_error":"","all_parents":"no_path","driver":"GTiff","ban
dIndex |
| ":"1","parentPath":"no_path", "last_command":"gdal_translate -of GTiff -b 1 -of GTiff
-co        |
| TILED=YES -co COMPRESS=DEFLATE"}}
|
+-----------------------------------------------------------------------------------------
---------+
```

`gbx_rst_skewx(tile)`

Computes the skew of the raster tile in the X direction.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: DoubleType

Example:

```
SQL
SELECT gbx_rst_skewx(tile) FROM table
```

```
+--------------------+
| gbx_rst_skewy(tile) |
+--------------------+
| 1.2                |
+--------------------+
```

`gbx_rst_skewy(tile)`

Computes the skew of the raster tile in the Y direction.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: DoubleType

Example:

```
SQL
SELECT gbx_rst_skewy(tile) FROM table
```

```
+--------------------+
| gbx_rst_skewy(tile) |
+--------------------+
| 1.2                |
+--------------------+
```

`gbx_rst_srid(tile)`

Returns the SRID of the raster tile as an EPSG code.

For complex CRS definitions the EPSG code may default to 0.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: IntegerType

Example:

```
SQL
SELECT gbx_rst_srid(tile) FROM table
```

```
+--------------------+
| gbx_rst_srid(tile) |
+--------------------+
| 9122               |
+--------------------+
```

`gbx_rst_subdatasets(tile)`

Returns the subdatasets of the raster tile as a set of paths in the standard GDAL format.

The result is a map of the subdataset path to the subdatasets and the description of the subdatasets.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: MapType(StringType, StringType)

Example:

```
SQL
SELECT gbx_rst_subdatasets(tile) FROM table
```

```
+-------------------------------------------------------------------------------------------
---------+
| gbx_rst_subdatasets(tile)
|
+-------------------------------------------------------------------------------------------
---------+
```

```
|  {
"NETCDF:\"/dbfs/FileStore/geospatial/mosaic/sample_raster_data/binary/netcdf-coral/ct5km_b
aa  |
| _max_7d_v3_1_20220106-1.nc\":bleaching_alert_area": "[1x3600x7200] N/A (8-bit unsigned
integer)" |
| ,
"NETCDF:\"/dbfs/FileStore/geospatial/mosaic/sample_raster_data/binary/netcdf-coral/ct5km_b
aa_m |
| ax_7d_v3_1_20220106-1.nc\":mask": "[1x3600x7200] mask (8-bit unsigned integer)"}
|
+-----------------------------------------------------------------------------------------
---------+
```

`gbx_rst_summary(tile)`

Returns a summary description of the raster tile including metadata and statistics in JSON format.

Values returned here are produced by the `gdalinfo` procedure.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: MapType(StringType, StringType)

Example:

```sql
SQL
SELECT gbx_rst_summary(tile) FROM table
```

```
+-----------------------------------------------------------------------------------------
---------+
| gbx_rst_summary(tile)
|
+-----------------------------------------------------------------------------------------
---------+
| {
"description":"/dbfs/FileStore/geospatial/mosaic/sample_raster_data/binary/netcdf-coral/ct
5km_ | | baa_max_7d_v3_1_20220106-1.nc",   "driverShortName":"netCDF",
"driverLongName":"Network Common | | Data Format",   "files":[
"/dbfs/FileStore/geospatial/mosaic/sample_raster_data/binary/netcdf-co |
| ral/ct5km_baa_max_7d_v3_1_20220106-1.nc" ],   "size":[     512,     512   ],
"metadata":{      |   | "":{       "NC_GLOBAL#acknowledgement":"NOAA Coral Reef    |
|Watch Program",                    | | "NC_GLOBAL#cdm_data_type":"Gr...
|
+-----------------------------------------------------------------------------------------
---------+
```

`gbx_rst_tooverlappingtiles(tile, tile_width, tile_height, overlap)`

Splits each `tile` into a collection of new raster tiles of the given width and height, with an overlap of `overlap` percent.

The result set is automatically exploded into a row-per-subtile.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **tile_width** (*Column (IntegerType)*) – The width of the tiles in pixels.
- **tile_height** (*Column (IntegerType)*) – The height of the tiles in pixels.
- **overlap** (*Column (IntegerType)*) – The overlap of the tiles in percentage.

Return type:

Column: Raster Tiles

Example:

```
SQL
SELECT gbx_rst_tooverlappingtiles(tile, 10, 10, 10) FROM table
```

```
+-------------------------------------------------------------------------------+
| gbx_rst_tooverlappingtiles(tile, 10, 10, 10)                                  |
+-------------------------------------------------------------------------------+
| {index_id: …, raster: [00 01 10 ... 00], parentPath: "...", driver: "GTiff" } |
| {index_id: …, raster: [00 03 10 ... 00], parentPath: "...", driver: "GTiff" } |
+-------------------------------------------------------------------------------+
```

`gbx_rst_transform(tile, target_srid)`

Transforms the raster to the given SRID.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **target_srid** (*Column (IntegerType)*) – EPSG authority code for the file's projection.

Return type:

Column: Raster Tile

Example:

```
SQL
SELECT gbx_rst_transform(tile,4326) FROM table
```

```
+-----------------------------------------------------------------------------------------
---------+
| gbx_rst_transform(tile,4326)
|
+-----------------------------------------------------------------------------------------
---------+
| {"index_id":null,"raster":"SUkqAAg...=
(truncated)","metadata":{"path":"...","last_error":"",    |
|  "all_parents":"no_path","driver":"GTiff","parentPath":"no_path",
|
|  "last_command":"gdalwarp -t_srs EPSG:4326 -of GTiff -co TILED=YES -co
COMPRESS=DEFLATE"}}       |
+-----------------------------------------------------------------------------------------
---------+
```

gbx_rst_tryopen(tile)

Tries to open the raster tile. If the raster cannot be opened the result is false, and if the raster can be opened the result is true.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: BooleanType

Example:

```
SQL
SELECT gbx_rst_tryopen(tile) FROM table
```

```
+----------------------+
| gbx_rst_tryopen(tile) |
+----------------------+
| true                  |
+----------------------+
```

gbx_rst_type(tile)

Returns the data type of the raster's bands.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: ArrayType[StringType]

Example:

```
SQL
SELECT gbx_rst_type(tile) FROM table
```

```
+--------------------+
| gbx_rst_type(tile) |
+--------------------+
| [Int16]            |
+--------------------+
```

`gbx_rst_updatetype(tile, new_type)`

Translates the raster to a new data type.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **new_type** (*Column (StringType)*) – Data type to translate the raster to.

Return type:

Column: Raster Tile

Example:

```
SQL
SELECT gbx_rst_updatetype(tile, 'Float32') FROM table
```

```
+-------------------------------------------------------------------------------------------
---------+
| gbx_rst_updatetype(tile,Float32)
|
+-------------------------------------------------------------------------------------------
---------+
| {"index_id":null,"raster":"SUkqAAg...=
(truncated)","metadata":{"path":"...","last_error":"",     |
|  "all_parents":"no_path","driver":"GTiff","parentPath":"no_path",
|
|  "last_command":"gdaltranslate -ot Float32 -of GTiff -co TILED=YES -co
COMPRESS=DEFLATE"}}       |
+-------------------------------------------------------------------------------------------
---------+
```

gbx_rst_upperleftx(tile)

Computes the upper left X coordinate of `tile` based on its GeoTransform.

Parameters:

- **tile** (*Column)* – Raster Tile.

Return type:

Column: DoubleType

Example:

```
SQL
SELECT gbx_rst_upperleftx(tile) FROM table
```

```
+-------------------------+
| gbx_rst_upperleftx(tile) |
+-------------------------+
| -180.00000610436345     |
+-------------------------+
```

gbx_rst_upperlefty(tile)

Computes the upper left Y coordinate of `tile` based on its GeoTransform.

Parameters:

- **tile** (*Column)* – Raster Tile.

Return type:

Column: DoubleType

Example:

```
SQL
SELECT gbx_rst_upperlefty(tile) FROM table
```

```
+-------------------------+
| gbx_rst_upperlefty(tile) |
+-------------------------+
| 89.99999847369712       |
+-------------------------+
```

gbx_rst_width(tile)

Computes the width of the raster tile in pixels.

Parameters:

- **tile** (*Column*) – Raster Tile.

Return type:

Column: IntegerType

Example:

```
SQL
SELECT gbx_rst_width(tile) FROM table
```

```
+--------------------+
| gbx_rst_width(tile) |
+--------------------+
| 600                |
+--------------------+
```

`gbx_rst_worldtorastercoord(tile, world_x, world_y)`

Computes the (j, i) pixel coordinates of `world_x` and `world_y` within `tile` using the CRS of `tile`.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **world_x** (*Column (DoubleType)*) – X world coordinate.
- **world_y** (*Column (DoubleType)*) – Y world coordinate.

Return type:

Column: Named StructType(IntegerType, IntegerType)

Example:

```
SQL
SELECT gbx_rst_worldtorastercoord(tile, -160.1, 40.0) FROM table
```

```
+-----------------------------------------------+
| gbx_rst_worldtorastercoord(tile, -160.1, 40.0) |
+-----------------------------------------------+
| {"x": 398, "y": 997}                          |
+-----------------------------------------------+
```

`gbx_rst_worldtorastercoordx(tile, world_x, world_y)`

Computes the (j, i) pixel coordinates of `world_x` within `tile` using the CRS of `tile`.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **world_x** (*Column (DoubleType)*) – X world coordinate.
- **world_y** (*Column (DoubleType)*) – Y world coordinate.

Return type:

Column: IntegerType

Example:

```
SQL
SELECT gbx_rst_worldtorastercoordx(tile, -160.1, 40.0) FROM table
```

```
+-----------------------------------------------+
| gbx_rst_worldtorastercoordx(tile, -160.1, 40.0) |
+-----------------------------------------------+
| 398                                           |
+-----------------------------------------------+
```

`gbx_rst_worldtorastercoordy(tile, world_x, world_y)`

Computes the (j, i) pixel coordinates of `world_y` within `tile` using the CRS of `tile`.

Parameters:

- **tile** (*Column*) – Raster Tile.
- **world_x** (*Column (DoubleType)*) – X world coordinate.
- **world_y** (*Column (DoubleType)*) – Y world coordinate.

Return type:

Column: IntegerType

Example:

```
SQL
SELECT gbx_rst_worldtorastercoordy(tile, -160.1, 40.0) FROM table
```

```
+-----------------------------------------------+
| gbx_rst_worldtorastercoordy(tile, -160.1, 40.0) |
+-----------------------------------------------+
| 997                                           |
+-----------------------------------------------+
```

VectorX Functions

Showing python import with SQL registration and notional example per function (refer to previous section for scala import). Since Product has introduced Vector support starting in DBR 14.2 and in Public Preview as of DBR 17.1 (see ST Geospatial Functions), this is intended to be a very short list of functions.

```python
from databricks.labs.gbx.vectorx.jts.legacy import functions as vx

vx.register(spark)
```

```sql
show functions like 'gbx_st_*'
```

`gbx_st_legacyaswkb(legacy_geom)`

Returns the legacy vector data format used by DBLabs Mosaic as WKB. This is intended for easy modernization for any data previously stored.

Parameters:

- **legacy_geom** (*Column)* – Legacy geometry format.

Return type:

Column: BinaryType

Example:

```sql
SELECT gbx_st_legacyaswkb(legacy_geom)
```

```
+-----------------------------------+
| gbx_st_legacyaswkb(legacy_geom) |
+-----------------------------------+
| [01 03 00 00 00 0...              |
| [01 03 00 00 00 0...              |
| [01 03 00 00 00 0...              |
+-----------------------------------+
```

GridX - BNG Functions

This is specific support for British National Grid (BNG). *Showing python import with SQL registration and notional example per function (refer to previous section for scala import).*

```python
from databricks.labs.gbx.gridx.bng import functions as bx

bx.register(spark)
```

```sql
show functions like 'gbx_bng_*'
```

`gbx_bng_aswkb(cellid)`

Returns grid cell boundary as a WKB.

Parameters:

- **cellid** (*Column: LongType | StringType)* – Index ID.

Return type:

Column: BinaryType

Example:

```sql
SELECT gbx_bng_aswkb(cellid)
```

```
+-------------------------+
| gbx_bng_aswkb(cellid) |
+-------------------------+
| [01 03 00 00 00 0...    |
| [01 03 00 00 00 0...    |
| [01 03 00 00 00 0...    |
+-------------------------+
```

`gbx_bng_aswkt(cellid)`

Returns grid cell boundary as a WKT.

Parameters:

- **cellid** (*Column: LongType | StringType)* – Index ID.

Return type:

Column: StringType

Example:

```
SQL
    SELECT gbx_bng_aswkt(cellid)
```

```
+------------------------+
| gbx_bng_aswkt(cellid) |
+------------------------+
| <wkt>                  |
| <wkt>                  |
| <wkt>                  |
+------------------------+
```

`gbx_bng_cellarea(cellid)`

Returns the area of a given cell in km^2.

Parameters:

- **cellid** (*Column: LongType | StringType*) – Index ID.

Return type:

Column: DoubleType

Example:

```
SQL
    SELECT gbx_bng_area(cellid)
```

```
+------------------------+
| gbx_bng_area(cellid) |
+------------------------+
|  0.78595419            |
|  1.419                 |
|  3.785                 |
+------------------------+
```

`gbx_bng_cellintersection(left_chip, right_chip)`

Returns the chip representing the intersection of two chips based on the same grid cell. Also, see gbx_bng_cellintersection_agg function.

The Named StructType is composed of:

- `core`: Identifies if the chip is fully contained within the geometry: Boolean
- `cellid`: Index ID: LongType | StringType
- `chip`: Geometry in WKB format equal to the intersection of the index shape and the original `geom`: BinaryType

Parameters:

- **left_chip** (*Column: StructType*) - Left Chip.
- **right_chip** (*Column: StructType*) - Right Chip.

Return type:

Column: StructType

Example:

```
SQL
   SELECT gbx_bng_cellintersection(left_chip, right_chip)
```

```
+-----------------------------------------------------+
| gbx_bng_cellintersection(left_chip, right_chip) |
+-----------------------------------------------------+
| {core: false, cellid: <id>, chip: ...}              |
| {core: false, cellid: <id>, chip: ...}              |
| {core: false, cellid: <id>, chip: ...}              |
+-----------------------------------------------------+
```

`gbx_bng_cellintersection_agg(chips)`

Aggregator that computes the chip representing the intersection of the input chips.

The Named StructType is composed of:

- `core`: Identifies if the chip is fully contained within the geometry: Boolean
- `cellid`: Index ID: LongType | StringType
- `chip`: Geometry in WKB format equal to the intersection of the index shape and the original `geom`: BinaryType

Parameters:

- **chips** (*Column*) – Chips.

Return type:

Column: StructType

Example:

```
SQL
    WITH chips AS (SELECT gbx_bng_tessellateexplode(geom) AS "chip" FROM ...)
    SELECT gbx_bng_cellintersection_agg(chips) AS agg_chip FROM chips GROUP BY
    chips.cellid;
```

```
+-------------------------------------+
| agg_chip                            |
+-------------------------------------+
| {core: false, cellid: <id>, chip: ...} |
| {core: false, cellid: <id>, chip: ...} |
| {core: false, cellid: <id>, chip: ...} |
+-------------------------------------+
```

`gbx_bng_cellunion(chip_1, chip_2)`

Returns the union of 2 chip_1 and chip_2. Also, see gbx_bng_cellunion_agg(chips) function.

The Named StructType is composed of:

- `core`: Identifies if the chip is fully contained within the geometry: Boolean
- `cellid`: Index ID: LongType | StringType
- `chip`: Geometry in WKB format equal to the intersection of the index shape and the original `geom`: BinaryType

Parameters:

- **chip_1** (*Column: StructType*) - Chip1.
- **chip_2** (*Column: StructType*) - Chip2.

Return type:

Column: StructType

Example:

```
SQL
SELECT gbx_bng_cellunion(chip_1, chip_2)
```

```
+--------------------------------------+
| gbx_bng_cellunion(chip_1, chip_2)  |
+--------------------------------------+
| {core: false, cellid: <id>, chip: ...} |
| {core: false, cellid: <id>, chip: ...} |
| {core: false, cellid: <id>, chip: ...} |
+--------------------------------------+
```

`gbx_bng_cellunion_agg(chips)`

Aggregator that computes the chip representing the union of the input chips.

The Named StructType is composed of:

- `core`: Identifies if the chip is fully contained within the geometry: Boolean
- `cellid`: Index ID: LongType | StringType
- `chip`: Geometry in WKB format equal to the intersection of the index shape and the original `geom`: BinaryType

Parameters:

- **chips** (*Column*) – Chips.

Return type:

Column: StructType

Example:

```
SQL
   WITH chips AS (SELECT gbx_bng_tessellateexplode(geom) AS "chip" FROM ...)
   SELECT gbx_bng_cellunion_agg(chips) AS agg_chip FROM chips GROUP BY
   chips.cellid;
```

```
+--------------------------------------+
| agg_chip                             |
+--------------------------------------+
| {core: false, cellid: <id>, chip: ...} |
| {core: false, cellid: <id>, chip: ...} |
| {core: false, cellid: <id>, chip: ...} |
```

```
+-------------------------------------+
```

`gbx_bng_centroid(cellid)`

Returns the centroid for a cellid Geometry.

Parameters:

- **cellid** (*Column: LongType | StringType*) – Index ID.

Return type:

Column: Geometry (WKB)

Example:

```
SQL
SELECT gbx_bng_centroid(cellid)
```

```
+--------------------------+
| gbx_bng_centroid(cellid) |
+--------------------------+
| [01 03 00 00 00 0...     |
| [01 03 00 00 00 0...     |
| [01 03 00 00 00 0...     |
+--------------------------+
```

`gbx_bng_distance(cellid1, cellid2)`

Returns the Manhattan Distance between two cellids.

Parameters:

- **cellid1** (*Column: LongType | StringType*) – Index ID.
- **cellid2** (*Column: LongType | StringType*) – Index ID.

Return type:

Column: LongType

Example:

```
SQL
SELECT gbx_bng_distance(cellid1, cellid2)
```

```
+-------------------------------------+
```

```
| gbx_bng_distance(cellid1, cellid2) |
+------------------------------------+
| 654                                |
| 4321                               |
| 767                                |
+------------------------------------+
```

`gbx_bng_eastnorthasbng(easting, northing, resolution)`

Returns the cellid for the given params.

Parameters:

- **easting** (*Column: DoubleType*) – easting.
- **northing** (*Column: DoubleType*) – northing.
- **resolution** (Column: IntegerType | StringType) - Index Resolution.

Return type:

Column: StringType

Example:

```
SQL
SELECT gbx_bng_eastnorthasbng(easting, northing, 5)
```

```
+-----------------------------------------------+
| gbx_bng_eastnorthasbng(easting, northing, 5) |
+-----------------------------------------------+
| <id>                                          |
| <id>                                          |
| <id>                                          |
+-----------------------------------------------+
```

`gbx_bng_euclideandistance(cellid1, cellid2)`

Returns the euclidean distance between two cellids.

Parameters:

- **cellid1** (*Column: LongType* | *StringType*) – Index ID.
- **cellid2** (*Column: LongType* | *StringType*) – Index ID.

Return type:

Column: LongType

Example:

```
SQL
SELECT gbx_bng_euclideandistance(cellid1, cellid2)
```

```
+-----------------------------------------------+
| gbx_bng_euclideandistance(cellid1, cellid2) |
+-----------------------------------------------+
| 654                                           |
| 4321                                          |
| 767                                           |
+-----------------------------------------------+
```

`gbx_bng_geometrykloop(geom, resolution, k)`

Returns the k-loop (hollow ring) of a given geometry.

Parameters:

- **geom** (*Column*) – Geometry (WKB | WKT).
- **resolution** (*Column: IntegerType | StringType*) – Index resolution.
- **k** (*Column: IntegerType*) – K-Loop size.

Return type:

Column: ArrayType(StringType)

Example:

```
SQL
    SELECT gbx_bng_geometrykloop(geom, 5, 2)
```

```
+-------------------------------------+
| gbx_bng_geometrykloop(geom, 5, 2) |
+-------------------+-----------------+
| [<id>, <id>, ...]                   |
+-------------------------------------+
```

`gbx_bng_geometrykloopexplode(geom, resolution, k)`

Generator that returns the k loop (hollow ring) of a given geometry exploded.

Parameters:

- **geom** (*Column*) – Geometry (WKB | WKT).
- **resolution** (*Column: IntegerType | StringType*) – Index resolution.
- **k** (*Column: IntegerType*) – K-loop size.

Return type:

Column: StringType

Example:

```
SQL
SELECT gbx_bng_geometrykloopexplode(geom, 5, 2) as kloop
```

```
+-------+
| kloop |
+-------+
| <id>  |
| <id>  |
| <id>  |
+-------+
```

`gbx_bng_geometrykring(geom, resolution, k)`

Returns the k-ring of a given geometry respecting the boundary shape.

Parameters:

- **geom** (*Column*) – Geometry (WKB | WKT).
- **resolution** (*Column: IntegerType | StringType*) – Index resolution.
- **k** (*Column: Integer*) – K-ring size.

Return type:

Column: ArrayType(StringType)

Example:

```
SQL
SELECT gbx_bng_geometrykring(geom, 5, 1)
```

```
+-----------------------------------+
| gbx_bng_geometrykring(geom, 5, 1) |
+-----------------------------------+
```

```
| [<id>, <id>, ...]                    |
+--------------------------------------+
```

`gbx_bng_geometrykringexplode(geom, resolution, k)`

Generator that returns the k-ring of a given geometry exploded.

Parameters:

- **geom** (*Column*) – Geometry (WKB | WKT).
- **resolution** (*Column: IntegerType | StringType*) – Index resolution.
- **k** (*Column: IntegerType*) – K-ring size.

Return type:

Column: StringType

Example:

```
SQL
SELECT gbx_bng_geometrykringexplode(geom, 5, 2) as kring
```

```
+-------+
| kring |
+-------+
| <id>  |
| <id>  |
| <id>  |
+-------+
```

`gbx_bng_kloop(cellid, k)`

Returns the k loop (hollow ring) of a given cell.

Parameters:

- **cellid** (*Column: LongType | StringType*) – Index ID.
- **k** (*Column: IntegerType*) – K-loop size.

Return type:

Column: ArrayType(StringType)

Example:

```
SQL
    SELECT cellid, gbx_bng_kloop(cellid, 2)
```

```
+-----------------------------------+
| cellid | gbx_bng_kloop(cellid, 2) |
+--------+--------------------------+
| <id>   | [<id>, <id>, ...]        |
+--------+--------------------------+
```

`gbx_bng_kloopexplode(cellid, k)`

Generator that returns the k loop (hollow ring) of a given cell exploded.

Parameters:

- **cellid** (*Column: LongType | StringType*) – Index ID.
- **k** (*Column: IntegerType*) – K-loop size.

Return type:

Column: LongType | StringType

Example:

```
SQL
SELECT cellid, gbx_bng_kloopexplode(cellid, 2) as kloop
```

```
+--------+-------+
| cellid | kloop |
+--------+-------+
| <id>   | <id>  |
| <id>   | <id>  |
| <id>   | <id>  |
+--------+-------+
```

`gbx_bng_kring(cellid, k)`

Returns the k-ring of a given cell.

Parameters:

- **cellid** (*Column: LongType | StringType*) – Index ID.
- **k** (*Column: IntegerType*) – K-ring size.

Return type:

Column: ArrayType[StringType]

Example:

```
SQL
SELECT cellid, gbx_bng_kring(cellid, 2)
```

```
+--------+---------------------------+
| cellid | gbx_bng_kring(cellid, 2) |
+--------+---------------------------+
| <id>   | [<id>, <id>, ...]          |
| <id>   | [<id>, <id>, ...]          |
| <id>   | [<id>, <id>, ...]          |
+--------+---------------------------+
```

`gbx_bng_kringexplode(cellid, k)`

Generator that returns the k-ring of a given cell exploded.

Parameters:

- **cellid** (*Column: LongType | StringType*) – Index ID.
- **k** (*Column: IntegerType*) – K-ring size.

Return type:

Column: LongType | StringType

Example:

```
SQL
SELECT cellid, gbx_bng_kringexplode(cellid, 1) as kring
```

```
+--------+-------+
| cellid | kring |
+--------+-------+
| <id>   | <id>  |
| <id>   | <id>  |
| <id>   | <id>  |
+--------+-------+
```

`gbx_bng_pointasbng(geom, resolution)`

Returns the `resolution` grid index associated with the input point geometry `geom`.

Parameters:

- **geom** (*Column*) – Geometry (WKB | WKT).
- **resolution** (*Column: IntegerType | StringType*) – Index resolution.

Return type:

Column: StringType

Example:

```
SQL
SELECT gbx_bng_pointasbng(geom, 1)
```

```
+----------------------------+
| gbx_bng_pointasbng(geom, 1) |
+----------------------------+
| <id>                       |
| <id>                       |
| <id>                       |
+----------------------------+
```

`gbx_bng_polyfill(geom, resolution)`

Returns the set of grid indices of which centroid is contained in the input `geom` at `resolution`.

Parameters:

- **geom** (*Column*) – Geometry (WKB | WKT).
- **resolution** (*Column: IntegerType | StringType*) – Index resolution.

Return type:

Column: ArrayType[StringType]

Example:

```
SQL
    SELECT gbx_bng_polyfill(geom, 1)
```

```
+--------------------------+
| gbx_bng_polyfill(geom, 1) |
+--------------------------+
```

```
| [<id>, <id>, ...]          |
+----------------------------+
```

`gbx_bng_tessellate(geom, resolution, <keep_core_geometries>)`

Cuts the original `geom` into several pieces along the grid index borders at the specified `resolution`.

Returns an array of Structs **covering** the input `geom` at `resolution`.

The return Named StructType is composed of:

- `core`: Identifies if the chip is fully contained within the geometry: Boolean
- `cellid`: Index ID: StringType
- `chip`: Geometry in WKB format equal to the intersection of the index shape and the original `geom`: BinaryType

In contrast to gbx_bng_tessellateexplode, this function does not explode the list of shapes.

In contrast to gbx_bng_polyfill, this function fully covers the original `geom` even if the index centroid falls outside of the original geometry. This makes it suitable to index lines as well.

Parameters:

- **geom** (*Column*) – Geometry (WKB | WKT).
- **resolution** (*Column: IntegerType | StringType*) – Index resolution.
- **keep_core_geometries** (*Column: BooleanType)* – Whether to keep the core geometries or set them to null, default true; **not adjustable with SQL bindings.**

Return type:

Column: ArrayType[StructType]

Example:

```
SQL
SELECT gbx_bng_tessellate(geom, 1)
```

```
+---------------------------------------+
| gbx_bng_tessellate(geom, 1)           |
+---------------------------------------+
```

```
| [{core: false, cellid: <id>, chip: ...} |
| [{core: false, cellid: <id>, chip: ...} |
| [{core: false, cellid: <id>, chip: ...} |
+----------------------------------------+
```

`gbx_bng_tessellateexplode(geom, resolution, <keep_core_geometries>)`

Generator that cuts the original `geom` into several pieces along the grid index borders at the specified `resolution`.

Returns the set of Structs **covering** the input `geom` at `resolution`.

The return Named StructType is composed of:

- `core`: Identifies if the chip is fully contained within the geometry: Boolean
- `cellid`: Index ID: StringType
- `chip`: Geometry in WKB format equal to the intersection of the index shape and the original `geom`: BinaryType

In contrast to gbx_bng_tessellate, this function generates one result row per chip.

In contrast to gbx_bng_polyfill, this function fully covers the original `geom` even if the index centroid falls outside of the original geometry. This makes it suitable to index lines as well.

Parameters:

- **geom** (*Column*) – Geometry (WKB or WKT).
- **resolution** (*Column: IntegerType | StringType)* – Index resolution.
- **keep_core_geometries** (*Column: BooleanType)* – Whether to keep the core geometries or set them to null, default true; **not adjustable with SQL bindings.**

Return type:

Column: StructType

Example:

```sql
-- -5 is "50m", see BNG.scala
SELECT gbx_bng_tessellateexplode(geom, -5) as tessellate
```

```
+----------------------------------------+
```

```
| tessellate                           |
+--------------------------------------+
| {core: false, cellid: <id>, chip: …} |
| {core: false, cellid: <id>, chip: …} |
| {core: false, cellid: <id>, chip: …} |
+--------------------------------------+
```

SQL

```sql
SELECT gbx_bng_tessellateexplode(geom, "50m") as tessellate
```

```
+--------------------------------------+
| tessellate                           |
+--------------------------------------+
| {core: false, cellid: <id>, chip: …} |
| {core: false, cellid: <id>, chip: …} |
| {core: false, cellid: <id>, chip: …} |
+--------------------------------------+
```